Trabajo de Iniciación a la Investigación: Study and application of central pattern generator circuits to the control of a modular robot

Fernando Herrero Carrón
Escuela Politécnica Superior, Universidad Autónoma de Madrid

August 30, 2007

Contents

1 Introduction **5**

2 Methodology **7**
2.1 CPGs to control robotic locomotion 7
2.2 Network models 8
2.2.1 Neuron model 8
2.2.2 Synapse model 9
2.2.3 Connecting neurons and synapses 11
2.3 The robot 12
2.3.1 Control of one module 12
2.4 CPG evaluation 14
2.4.1 Simulator 17
2.4.2 Initial conditions 17
2.4.3 Diagrams 17
2.4.4 Phase difference and period analysis 18
2.5 Simulating networks of neurons 20
2.5.1 Advantages of the library 20
2.5.2 Implementation 21
2.5.3 Wrapping rather than inheriting 21
2.5.4 The constructor parameters problem 22

3 Intermodule coordination **25**
3.1 Uncoupled modules 25
3.1.1 Motivation 25
3.1.2 Parameters 26
3.2 Symmetrical inhibition, weak coupling 28
3.2.1 Motivation 28
3.2.2 Diagram 28
3.2.3 Parameters 28
3.2.4 Evaluation 29
3.3 Symmetrical inhibition with strong coupling 32
3.3.1 Irregular activity 32
3.3.2 Parity synchronization mode 1 32

3.3.3 Parity synchronization mode 2 . . . 33
3.3.4 Evaluation . . . 34
3.4 Inhibitory loop . . . 42
3.4.1 Motivation . . . 42
3.4.2 Parameters . . . 42
3.4.3 Evaluation . . . 43
3.5 Push-pull model . . . 51
3.5.1 Motivation . . . 51
3.5.2 Parameters . . . 51
3.5.3 Evaluation . . . 51
3.6 Bistable intermediate neurons . . . 55
3.6.1 Motivation . . . 55
3.6.2 The bistable neuron model with instantaneous state transtition . . . 56
3.6.3 The bistable neuron model with delayed state transition . . . 58
3.6.4 Counteracting border effects . . . 59
3.6.5 Evaluation . . . 60

4 Discussion **63**
4.1 CPG summary . . . 63
4.1.1 Features . . . 65
4.1.2 Complexity . . . 65
4.2 Symmetry . . . 66
4.3 Coupling strength . . . 66
4.4 Coupling sign . . . 67
4.5 Topology . . . 67

5 Future work **69**

A A C++ library for simulation of dynamical networks **71**
A.1 Using synapses . . . 72
A.2 Simulating iterated maps . . . 73
A.3 Adding new models . . . 74
A.4 Available wrappers . . . 75
A.4.1 SystemWrapper . . . 75
A.4.2 IntegratedSystemWrapper . . . 75
A.4.3 TimeWrapper . . . 76
A.4.4 SerializableWrapper . . . 77

1

Introduction

Locomotion is an ability inherent to the animal kingdom. Throughout evolution, all different species have diversified and adapted the functions and shapes of their motor organs. This has unavoidably lead to a great variability in motor control neural systems [1]. However, despite the large number of solutions nature has come up with for the many species, their study reveals common principles, for which reason knowledge about *inferior* species can be carried to *higher* ones.

One characteristic common to every species is a hierarchical organization of the motor system. From the more cognitive tasks like the animal's will to move, the selection of a concrete movement and the planning of a sequence of actions, to the activation of a muscle, a number of independent functional modules coordinate to generate a coherent and effective activity pattern. So, for instance, the movement of a limb implies a coordination mechanism for the whole organ and delegation in control subsystems for each segment.

The goal of locomotion is to displace the animal in its environment. Therefore sensory input is fundamental for an effective adaptation. The locomotion system can be described as a closed loop control system [2], in which actions must always comply with the frame imposed by external conditions [3]. Experimental isolation of the sensory system and the motor system provides us with evidence that certain activities suffer important degradation, to the point of completely disappearing in the absence of feedback [1]. The importance of understanding the role of sensory activity in the generation of movement is therefore crucial.

Nevertheless, there exist certain components that survive despite absence of this feedback. Rhythmic movements generated by *central pattern generators* (CPGs) are an example.

CPGs are neural circuits that autonomously generate rhythmic activity to control motor neurons and are involved in plenty of the exercises that require periodicity, robustness and/or precision. In them, neurons activate at different phases of the rhythmic cycle. The main characteristics of rhythms generated by this kind of circuits are stability, flexibility and robustness [4].

Mechanisms giving rise to rhythmic activity are manifold: on one hand, the sort of activity of each neuron (regular spiking, bursting, chaotic bursting, etc.), and on the other

hand the interconnection among them [5, 6]. Topology's role is fundamental to achieve this coordination of the different components in the network. Depending on connectivity parameters, such phenomena can appear as frequency locking, phase or antiphase locking or rhythm regularization. Through adequate connectivity, neurons with regular activity can also be made to originate collective chaotic activity [7], or irregular neurons to collectively display a richer and more stable activity than otherwise [8].

There exists evidence that justifies the existence of excitatory and inhibitory synapses. Apparently, inhibitory synapses enable the aforementioned types of coordination [8, 9, 10].

The classical approach to robot control is to model the cynematics of the robot and then design a controller to especifically act upon its joints [11]. This implies taking into account all the possible situations that the robot may encounter and *program them in*. Thus, the robot's behaviour is constrained by design.

On the other hand, nature itself displays a huge range of behaviour, even in previously unseen environments. CPGs may not be designed with every possible input in mind, yet their flexibility allows for different emergent responses.

In this respect chaotic activity is a very important asset. Surprisingly, rich dynamics like those displayed by chaotic neurons allows for a larger portion of the state space of the system to be searched, and is a key element in finding optimal solutions [12]. That a neuron can exhibit chaotic activity does not necessarily mean that its routine behaviour be irregular. In fact, chaos usually only appears under certain combinations of parameters.

Based on these investigations, I study the use of GPGs to control and coordinate the robot described in [13]. It is a modular robot, very suitable for this work because of its worm like behaviour. Modularity is a key principle, in which a neuronal control model fits perfectly.

The goal of this work is to explore different configurations of CPGs that could drive this modular worm in a stable and regular manner, to extract common principles from them and determine what properties (invariants) must be present for a given configuration to be a good controller.

2

Methodology

In this chapter, the methodology of the work is described. We begin with a short motivation for the use of CPGs in robotic control, including a review of their main properties. Next, the mathematical models of neurons and synapses used to implement these CPGs are presented. Following this, we explain the methods used to evaluate each CPG, and the diagrams presented in the next chapter. To conclude, we describe the software library that has been created, used in all experiments.

2.1 CPGs to control robotic locomotion

Robots cover a wide range of applications: car assembling, assisted surgery, defense, entertainment... The main common characteristic is: they move in an automated manner. What sets robots appart from other "moving" machines? A car is not a robot because it must be directly controlled be a human, so it is not just movement that defines robots, but automated control of movement.

Again, depending on application field, the requirements of this control can be diverse. In very constrained settings like assembly lines, a simple sequence of fixed targets will do, but other environments may be more demanding.

Unpredictability in these requirements is a major problem when designing robots: unpredictable goals and unpredictable perturbations must be taken into account. But it is a problem that nature has already solved.

Central Pattern Generators (CPGs) are circuits of neurons that drive muscle activity (through specific neurons called "motoneurons") [1], and are very good candidates for automated movement control under these uncertain conditions. When working freely they are able to generate a stable activity (depending on design it can be regular or irregular), and they can recover from perturbations due to its attractive nature. Of course CPGs must be specially designed for a specific target.

There are already some works applying neuroscientific knowledge to robotic control ([14, 15, 16, 17], to name a few). All of them are concerned with the generation and coordination of various cyclic movements, e.g., legs in walking robots and modules in

lamprey or worm robots. The approach is to couple a nonlinear oscillator to each one of these parts and include some term in the oscillators' equations to simulate synaptic connections.

These systems are very stable and robust because of the attracting nature of the oscillators. However, while this is a very interesting approach, it is very simplistic. By using systems with rich dynamics like bursting neurons and diffusion synapses we augment the circuit's information capacity. We plan to take advantage of this by implementing sensory feedback into the robot in a future work.

2.2 Network models

Usually models of neurons are a mathematical description of their physical properties. Making use of differential equations, one can describe the dynamics of ion conductances, of membrane potential and any other aspect subject to change with time [18].

Since solving these differential equations is usually not feasible, one has to resort to simulating a solution through integration algorithms. They are usually computationally costly because they imply speculating about possible future states of the system, evaluating the equations many times and interpolating the speculated results.

Map models make a more phenomenological description of neurons [19, 20]. Their purpose is to mimic neurons' behaviour without necessarily giving an explanation of why or how this behaviour happens.

They can be considered as "discretized time" differential equations. Eventhough still hard to solve and in most cases they do not have an analytical solution, the next state can be exactly computed given the current one, as opposed to continuous differential equations. Discrete maps are of the form: $x_{n+1} = f(x_n)$, so computing the next state involves evaluating f only once, much more efficient than continuous integration algorithms.

2.2.1 Neuron model

All neurons of the CPGs used in this study are based on Rulkov's model [21]. Rulkov's model was chosen because of its simplicity and the rich set of possible behaviours depending on the selection of parameters. The mathematical description of this model is shown in equations 2.1, 2.2 and 2.3.

$$f(x,y) = \begin{cases} \frac{\alpha}{1-x} + y & \text{if } x \leq 0 \\ \alpha + y & \text{if } 0 \leq x < \alpha + y \\ -1 & \text{otherwise} \end{cases} \tag{2.1}$$

$$x_{n+1} = f(x_n, y_n + \beta_e I_n) \tag{2.2}$$

$$y_{n+1} = y_n - \mu(x_n + 1) + \mu\sigma + \mu\sigma_e I_n \tag{2.3}$$

This is a bidimensional model, where variable x_n represents a neuron's membrane voltage and y_n is a slow dynamics variable with no direct biological meaning, but with

similar meaning as *gating* variables in biological models that represent the fraction of open ion-channels in the cell. So to speak, while x_n oscillates on a fast time scale, representing individual spikes of the neuron, y_n keeps track of *where* in time the burst finds itself, a sort of context memory.

Two parameters, σ and α select the working regime of the model: silent, tonic spiking or tonic bursting. In the most interesting regime for this work, tonic bursting, they furthermore control burst duration and period, so experimenting with these should be straightforward.

Figure 2.1 shows the effect of changing α and σ in the tonic bursting regime[1]. *Grosso modo*, α controls the burst width and σ controls interburst intervals (while also affecting the duration of the burst).

Since this model is phenomenological, we will not be attaching units to the plots. It should be undertood that abscissas will represent time evolution and ordinates membrane potential.

An undesirable side effect of parameter α is the change of amplitude of the spikes as in Figure 2.1, where a change from $\alpha = 6$ to $\alpha = 15$ causes a change in spike amplitude from approximately 4 a.u. to approximately 14 a.u. Every time this parameter is changed in a network, synapses must be readjusted since the new amplitude might be larger than the synapse's reversal potential. More on this in the next section.

2.2.2 Synapse model

Synaptic models are based on Destexhe's model [22]. To sum up, they model chemical synapses that release a neurotransmitter during a certain amount of time after the presynaptic neuron has crossed a given threshold value for its membrane potential.

Equation 2.4 is the equation that defines the ratio of bound chemical receptors in the postsynaptic neuron[2], where r is the fraction of bound receptors, α and β are the forward and backward rate constants for transmitter binding and $[T]$ is neurotransmitter concentration. The condition on time specifies that during ρ units of time after the presynaptic neuron's membrane potential last crossed[3] threshold τ (moment t_f), there is a concentration $[T]$ of neurotransmitter in the synaptic cleft that can be bound to free receptors in the postsynaptic neuron. After ρ units of time, there is no more release of neurontransmitter, thus the only term affecting r is the *unbounding* of already bounded receptors.

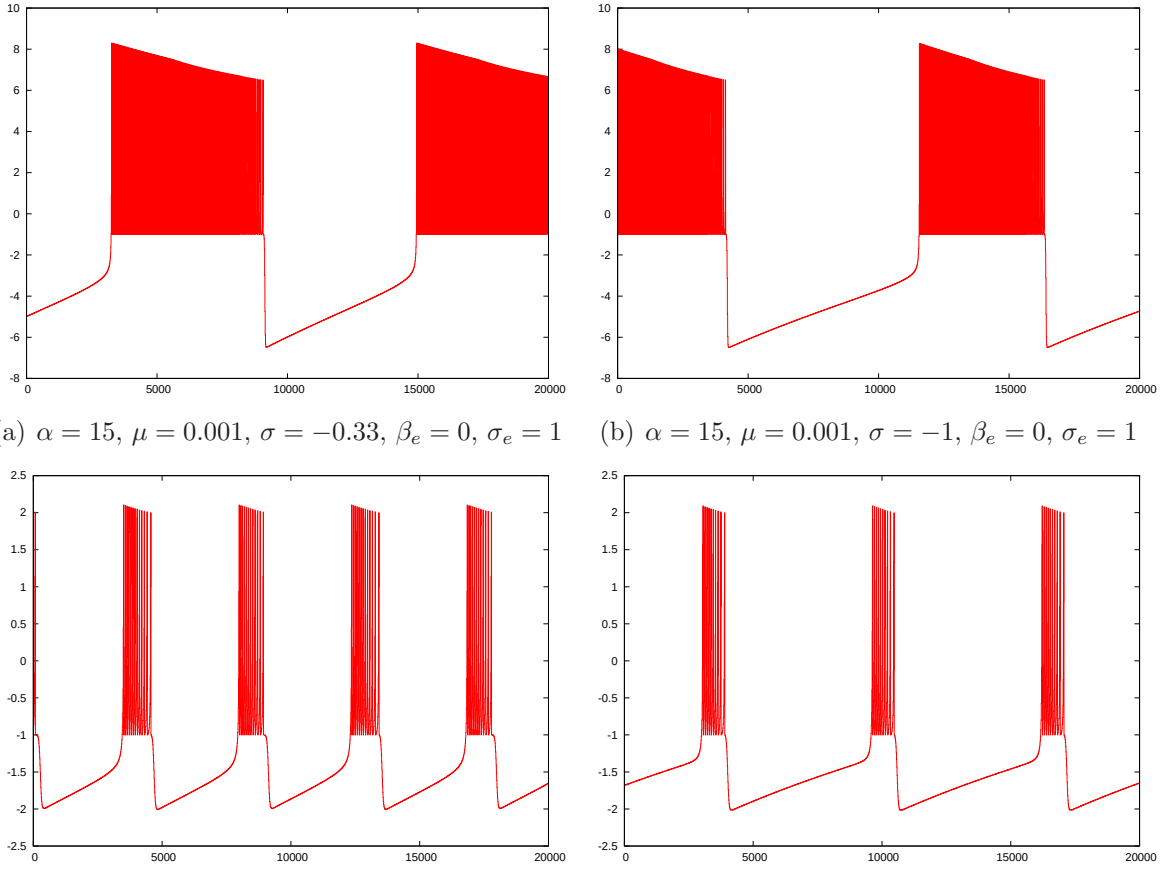
$$\dot{r} = \begin{cases} \alpha[T](1-r) - \beta r & \text{if } t_f < t < t_f + \rho \\ -\beta r & otherwise \end{cases} \tag{2.4}$$

In this work we set this threshold to be at $\tau = 0$, so that each single spike within a burst will cause neurotransmitter release at a constant concentration $[T] = 1$ for $\rho = 0.01$

[1] For an explanation of all possible regimes see [21]

[2] we will sometimes call it "synaptic activation"

[3] such that $(V_t < \tau)$ & $(V_{t+dt} \geq \tau)$



(a) $\alpha = 15$, $\mu = 0.001$, $\sigma = -0.33$, $\beta_e = 0$, $\sigma_e = 1$

(b) $\alpha = 15$, $\mu = 0.001$, $\sigma = -1$, $\beta_e = 0$, $\sigma_e = 1$

(c) $\alpha = 6$, $\mu = 0.001$, $\sigma = -0.33$, $\beta_e = 0$, $\sigma_e = 1$

(d) $\alpha = 6$, $\mu = 0.001$, $\sigma = -0.45$, $\beta_e = 0$, $\sigma_e = 1$

Figure 2.1: *Effect of parameters α and σ in Rulkov's map: for a greater value of α, bursts get longer ((a) and (b) compared to (c) and (d)), with more spikes; for a lower value of σ, interburst time gets longer ((b) and (d) compared to (a) and (c)). These values are restricted to a region where the model shows tonic bursting; other combinations may give rise to tonic spiking, silent or chaotic behaviours.*

units of time and $[T] = 0$ the rest of the time.

Synaptic current is then calculated as in equation 2.5, where $I(t)$ is synaptict current at time t, g is synaptic conductance, $r(t)$ is the fraction of bound receptors at time t, $V_{post}(t)$ is the postsynaptic neuron's membrane potential and E_{syn} its reversal potential. It is called reversal potential because the direction of the current will change depending on whether V_{post} is greater or less than E_{syn}.

$$I(t) = g \cdot r(t) \cdot (V_{post}(t) - E_{syn}) \tag{2.5}$$

E_{syn} controls whether a synapse is excitatory or inhibitory. If E_{syn} is always smaller than the postsynaptic neuron's potential, then $I(t)$ will always be positive, thus the synapse is excitatory. If, on the other hand, E_{syn} is always greater than V_{post}, it will be

inhibitory. For this reason the reversal potential of synapses must be readjusted every time parameter α is changed in a postsynaptic neuron, since changing α also changes the amplitude of the postsynaptic membrane potential and this may change the intended effect of the synapse.

2.2.3 Connecting neurons and synapses

Let's put the two models together in a very simple network consisting of two neurons and one synapse from one to the other like shown in Figure 2.2.

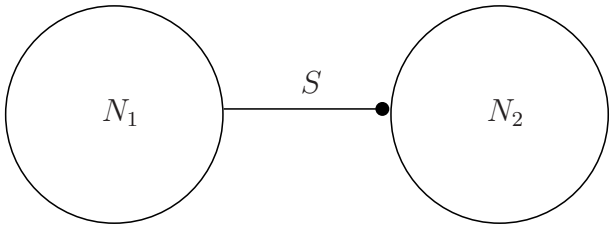


Figure 2.2: *simple network of two neurons, one (N_1) inhibiting the other (N_2) through an inhibitory synapse (S).*

To obtain the full equations of the network we must substitute the generic terms for the actual ones. Let's do it for all three "objects" of the network.

Since N_1 does not have any input, we let $I_n = 0$ in 2.2 and 2.3, thus its equations become:

$$x_{n+1}^{(1)} = f(x_n^{(1)}, y_n^{(1)}) \tag{2.6}$$
$$y_{n+1}^{(1)} = y_n^{(1)} - \mu(x_n^{(1)} + 1) + \mu\sigma \tag{2.7}$$

For S, we must substitute the time when N_1 last crossed S's threshold and the postsynaptic potential of N_2: we substitute t_f for $n_f^{(1)}$ and V_{post} for $x_n^{(2)}$ in equations 2.4 and 2.5:

$$\dot{r} = \begin{cases} \alpha[T](1-r) - \beta r & \text{if } n_f^{(1)} < t < n_f^{(1)} + \rho \\ -\beta r & otherwise \end{cases} \tag{2.8}$$
$$I(t) = g \cdot r(t) \cdot (x_n^{(2)} - E_{syn}) \tag{2.9}$$

We now have quantities defined for continuous time mixed with ones defined in discrete time. We need to discretize $I(t)$ and $r(t)$ in order for the equations to make sense.

In order to compute $I(t)$ and $r(t)$, we use an algorithm of the Runge-Kutta family, which defines sequences I_n and r_n, much like a sampling process. The algorithm takes as input a time step that specifies the time interval between sampling I_n and I_{n+1}. The

shorter the interval, the more precise the approximation, but the more steps will be needed to simulate a given amount of time.

In this work, synapses are simulated ten times more often than neurons. The reason is that using a smaller time step and higher evaluation frequency we achieve a smoother behaviour. The equations could be finally written as:

$$\begin{aligned} r_n^S &= n^{th}\ integration\ step\ of\ the \\ &\quad Runge-Kutta\ algorithm\ for\ \dot{r} && (2.10) \\ I_n^S &= g \cdot r_n^S \cdot (x^{(2)}_{\lfloor n/10 \rfloor} - E_{syn}) && (2.11) \end{aligned}$$

To obtain the equations for N_2, we must only substitute the term for the input current:

$$\begin{aligned} x^{(2)}_{n+1} &= f(x_n^{(2)}, y_n^{(2)} + \beta_e I^S_{10n}) && (2.12) \\ y^{(2)}_{n+1} &= y_n^{(2)} - \mu(x_n^{(2)} + 1) + \mu\sigma + \mu\sigma_e I^S_{10n} && (2.13) \end{aligned}$$

Figures 2.3 and 2.4 show how synapse S in Figure 2.2 behaves depending on the pre- and postsynaptic neurons. Figure 2.3 shows how when N_1 fires, the synapse S releases neurotransmitters that bind to N_2 receptors (higher value of r). The higher N_1's frequency is, the higher the synaptic activation. As the figure shows, spike frequency decreases as the burst gets closer to its end, and so does synaptic activation too. Figure 2.4 shows the resulting inhibitory synaptic current and its effect on N_2. The activation of the synapse produces a negative current that terminates the burst earlier and causes a longer interburst interval.

2.3 The robot

The robot for which a control system is being designed is introduced in [13]. It consists of several modules attached one after another through special connection points. Each of these modules consists of two triangle shaped rigid pieces, joint by one vertex of the triangle, and a servomotor controlling the angle between these two pieces. Figure 2.5 shows a diagramatic representation of the actual robot shown in Figure 2.6.

In this work, a robot of ten modules has been considered. The parity of the number of modules may have an effect on some CPG models, but it has not been investigated.

2.3.1 Control of one module

The first goal is to establish a central pattern generator (CPG) that can drive a single module. We propose a very simple circuit that generates an oscillatory activity.

The organization of the module is shown in Figure 2.7(a). There are two endogenously rhythmic neurons (R and P) interconnected with inhibitory synapses. The role of

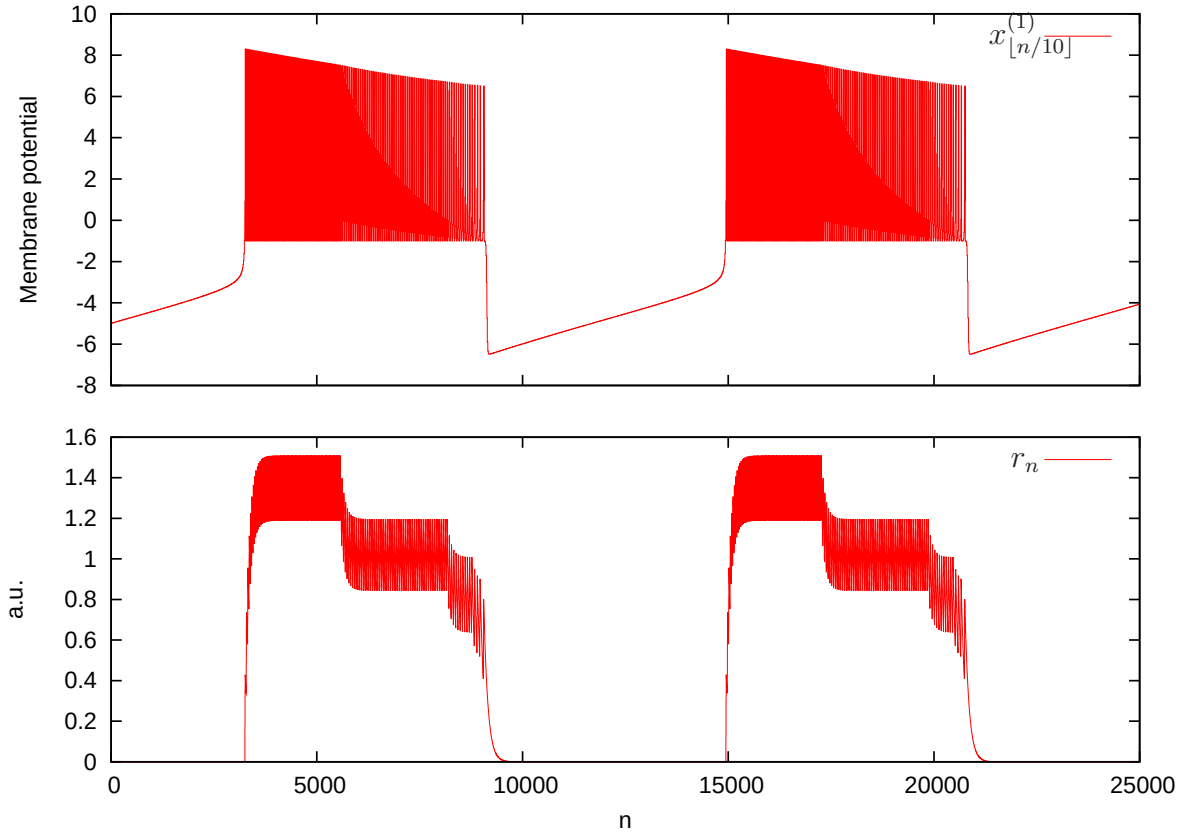


Figure 2.3: *Presynaptic neuron membrane potential and synaptic activation in the postsynaptic neuron (variable r in [22], scaled by a factor of 100). Synaptic activity closely follows neural activity, although there is a noticeable delay between the onset of the burst and the highest synaptic rate value. The three different stages of synaptic activation correspond to three different frequencies of the presynaptic neuron. Synapse parameters:* $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = 9$, $g_{syn} = 1.5$, $[T]_{max} = 1$ *and release time* $= 0.01$. *Neuron parameters:* $\alpha = 15$, $\mu = 0.001$, $\beta_e = 0$, $\sigma_e = 1$ *and* $\sigma = -0.33$.

inhibition is to prevent both neurons from firing at the same time. When one fires, the other's activity is delayed. In turn, when the second neuron is bursting, it delays the first neuron's activity and the cycle begins again. After some cycles, the two neurons arrive at a steady anti-phase state. These neurons are connected to the motoneuron (M) with a very simple synapse consisting only of a threshold function (eq. 2.14).

The motor neuron integrates its input and produces a signal that directly drives the motor, according to equation 2.15, where m is the output of neuron (M), the s terms are input from (R) and (P) and τ is a time constant that controls how quick the output signal m will change.

Due to the opposed contribution from the s term from (R) and the s term from (P) (opposite sign for the α parameter), and the fact that (P) and (R) oscillate in anti-phase, the solution $m(t)$ is a function oscillating between $-\alpha$ and α. When the motoneuron

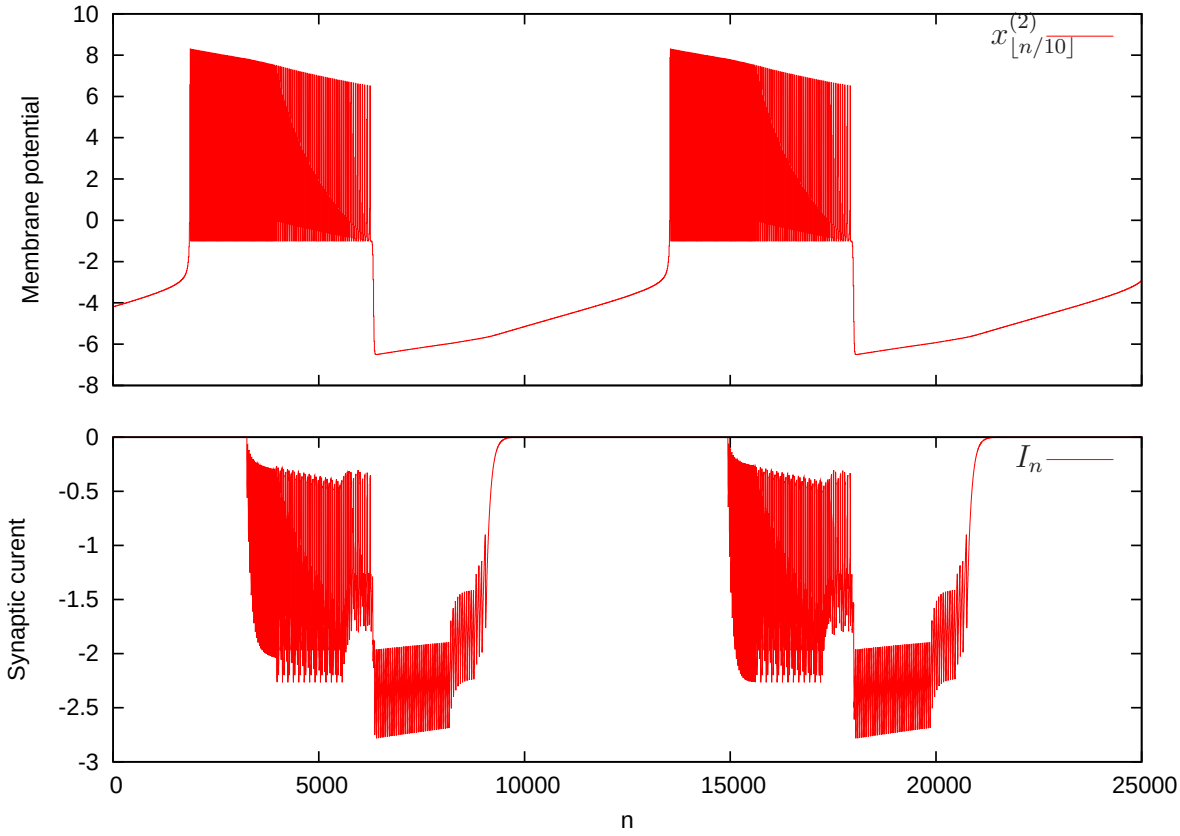


Figure 2.4: *Postsynaptic neuron membrane potential and synaptic current (variable I in [22], scaled by a factor of 10). Note how, due to the effect of the inhibitory synapse, the burst is shorter and the time between bursts is larger than in the presynaptic neuron, despite having the same parameters. Synapse parameters:* $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = 9$, $g_{syn} = 1.5$, $[T]_{max} = 1$ *and release time* $= 0.01$. *Neuron parameters:* $\alpha = 15$, $\mu = 0.001$, $\beta_e = 0$, $\sigma_e = 1$ *and* $\sigma = -0.33$.

receives no input because (P) and (R) are silent, it will go back to zero due to the $-m$ term in eq. 2.15.

Figure 2.7(b) shows an example pattern of activity of an isolated module.

$$s_{(\alpha,t)}(x) = \begin{cases} \alpha & \text{if } x > t \\ 0 & \text{otherwise} \end{cases} \tag{2.14}$$

$$\tau \dot{m} = -m + s_{(\alpha,t)}(R_x) + s_{(-\alpha,t)}(P_x) \tag{2.15}$$

2.4 Central pattern generator evaluation

In this section we explain the methods used to evaluate the CPGs as well as the diagrams and plots used in the following chapter, where we discuss the different proposed CPGs

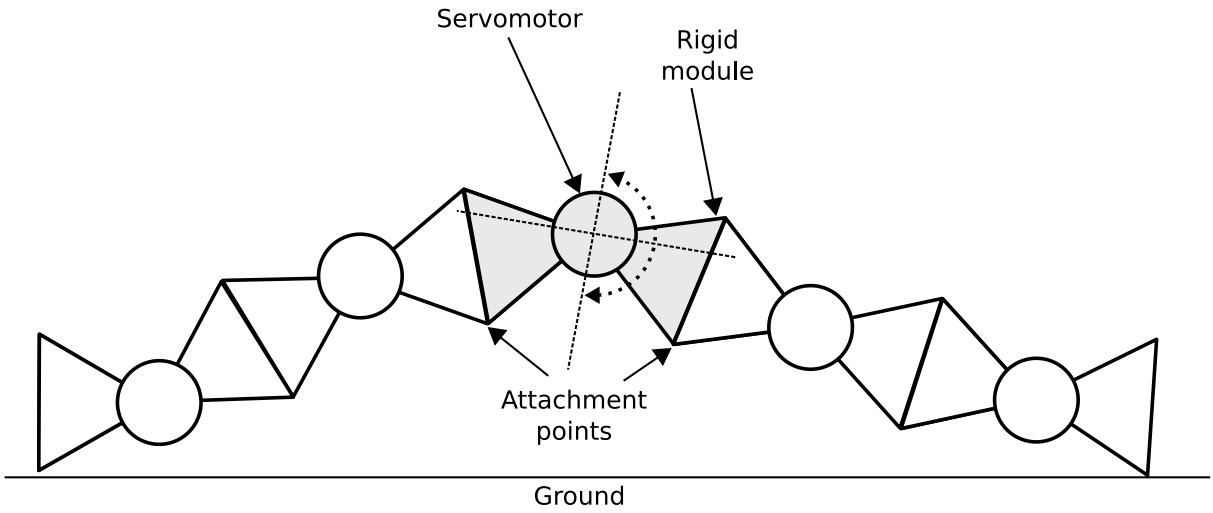


Figure 2.5: *General schema of the worm robot. A single module is marked in grey, all other modules are exactly equal to this one.*

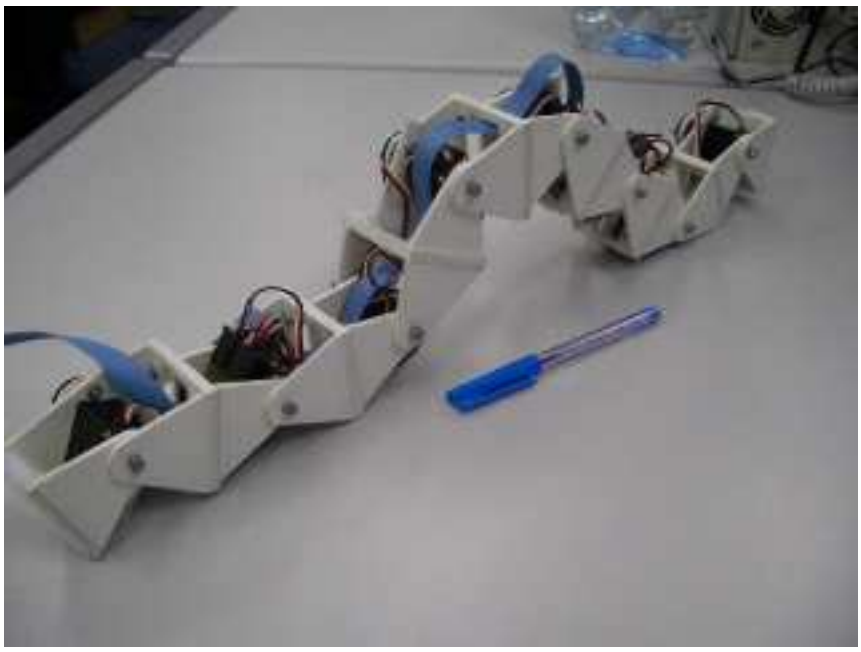
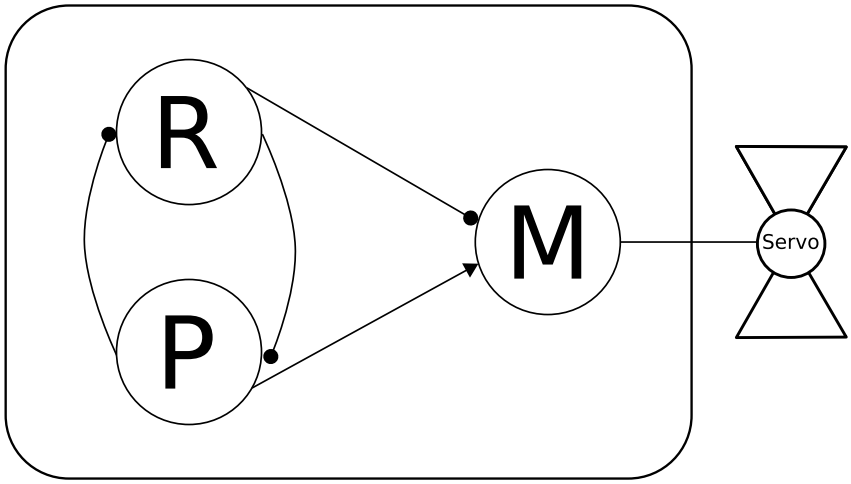
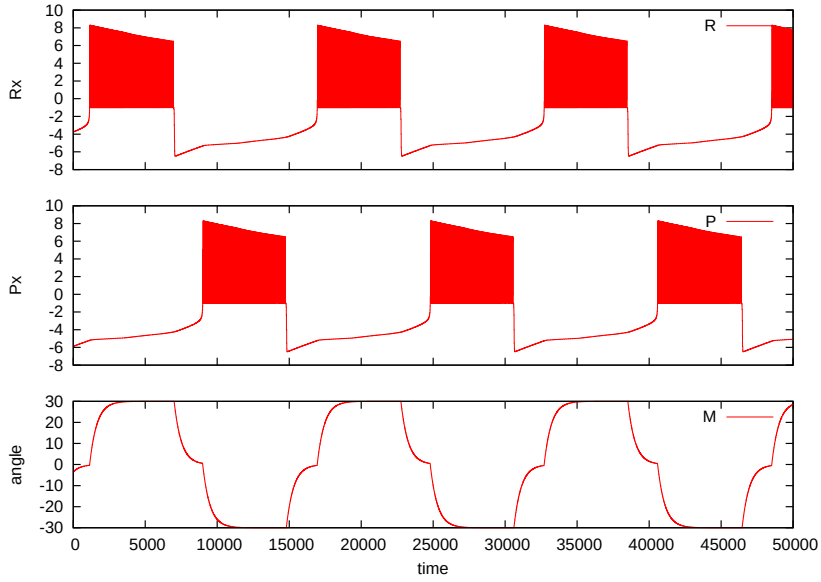


Figure 2.6: *Picture of the actual robot (taken with permission from http://www.iearobotics.com).*



(a)



(b)

Figure 2.7: *(a) Organization of the CPG within one module. The promotor 'P' and remotor 'R' neurons are interconnected with inhibitory synapses so that they synchronize in anti-phase. The motoneuron 'M' sends a command signal to the servomotor specifying the angle at which the servo should position itself. The signal generated by 'P' is directly input to 'M', and the opposite of the signal generated by 'R' is input to 'M'. 'M' integrates its input according to equation 2.15. (b)Activity sample. Neuron 'R' contributes positively and raises M to 30; neuron 'P' does exactly the opposite and drives 'M' towards -30. 'P' and 'R' are synchronized in anti-phase. With no input, 'M' goes to zero. Parameters for 'P' and 'R': $\alpha = 15$, $\mu = 0.001$, $\sigma = -0.33$, $beta_e = 0$, $\sigma_e = 1$; parameters for the inhibitory synapses between 'P' and 'R': $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = 9$, $g_{syn} = 1.5$, $T = 1$, $release_time = 0.01$; Parameters for 'M': $\alpha = 30$, $\tau = 0.5$, $t = -1.5$.*

for the control and coordination of the robot.

2.4.1 Simulator

All models have been implemented and tested on a rigid body dynamics simulator. The original simulator was developed by Juan González[4] and Rafael Treviño and has been adapted to support the CPGs of this work.

The simulator places a virtual modular robot in a virtual world and simulates gravity, friction and inertia with help of the ODE[5] physics engine.

2.4.2 Initial conditions

At the beginning of a simulation neurons are set to a known state. For random initial conditions, every neuron is iterated a random number of times before the simulation. This brings neurons into a random but steady state.

Synapses start with its 'binding rate' variable r set to zero.

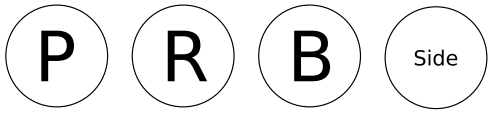
2.4.3 Diagrams

For every model CPG proposed there will be an explanatory diagram showing how modules are interconnected. Except where explicitly noted, all topologies will have repetition, meaning all modules will have the same interconnection scheme. Therefore some diagrams will only show the topology for two adjacent modules. It should be understood that the same scheme is repeated for the rest of the modules. Sometimes there is no possibility of repeating the whole scheme at the borders. In these cases the scheme is repeated as far as it is possible.

When diagrams are too complicated, only one side may be shown (either promotors or remotors only). This implies that the other side has the same connectivity pattern.

Neurons

Neurons will be represented with a white circle labelled 'P' for promotor neurons, 'R' for remotor neurons, 'B' for "bistable" neurons and 'side' for a regular neuron in the "inhibitory loop" CPG (both on the promotor and the remotor sides):



[4]http://www.iearobotics.com
[5]http://www.ode.org

Synapses

Synapses will be finished with a filled triangle when they are excitatory and with a filled circle when they are inhibitory:



2.4.4 Phase difference and period analysis

In order to capture the relevant aspects of the evolution in time of a given central pattern generator circuit, we provide two diagrams: time evolution of intermodule phase differences and time evolution of bursting period.

The diagrams included are extracted from **one** simulation, they are not in any way the mean behaviour, but they are chosen so that they are representative enough. Defining what is representative is done by direct observation of both the diagrams and behaviour in the simulator.

The length of one period is calculated as the number of iterations from the beginning of one burst to the beginning of the next one. The beginning of one burst is determined as the moment at which a signal crosses the threshold value $\tau = -1.2$ upwards. The Rulkov map resets intraburst spikes to a fixed value of -1, so in a "normal" regime neurons should only meet this condition when initiating a burst.

It is possible that a neuron remaining in underthreshold oscillation crosses this value of $\tau = -1.2$, but every model was inspected for this phenomenon and it did not appear in any of the examples shown.

Phase difference diagrams

Phase difference between modules is a key aspect of sequence generation.

We provide diagrams showing the time evolution of phase differences between every three consecutive modules, taking the first neuron as the reference. As an example, Figure 2.8 shows how phase differences ϕ_1 and ϕ_2 are calculated between N_1 and N_2 and N_1 and N_3.

Phase difference evolution is usually smooth. There are some cases when one neuron overtakes another one and this appears as a straight line crossing the diagram from 0 to 1 or from 1 to 0. This is not a discontinuity, but rather an effect of the modular arithmetic properties of phase.

Bursting period diagrams

Phase difference conveys a very meaningful information, but it might happen that while phase difference between two neurons is maintained constant, both neurons may be equally changing their periods. Therefore both phase difference and period informations are complementary.

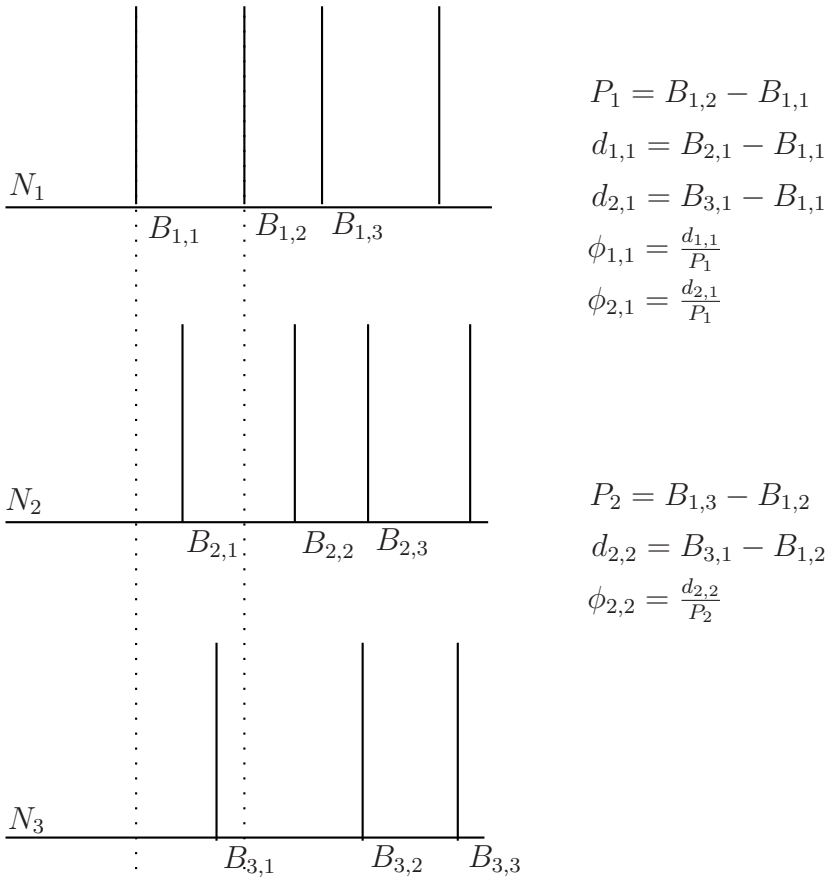


Figure 2.8: *An example of how phase differences are calculated. $B_{i,j}$ represents the beginning of the $j-th$ burst of neuron i. We take neuron N_1 as the reference and measure its first period, $P1$. We record the times at which the other two have last fired before $B_{1,2}$ and measure the difference between those times and the beginning of the period: $d_{1,1}$ and $d_{2,1}$. Phase difference between N_1 and N_2 is calculated as $\phi_{1,1} = \frac{d_{1,1}}{P_1}$, and likewise $\phi_{2,1}$ for N_1 and N_3. We now take the next burst of N_1 and repeat the operation. Notice that our algorithm takes the last burst of the other two neurons before $B_{1,3}$, which might occur in time prior to $B_{1,2}$, as is the case for $B_{3,1}$. In this case, our algorithm would yield a negative value for the phase difference $\phi_{2,2}$.*

Period diagrams are calculated in a similar manner to phase differences diagrams: for every neuron we calculate the time t_n at which the nth burst crosses the threshold value $\tau = -1.2$ and the next time t_{n+1} when the next burst crosses the threshold again. The value of period p_{n+1} is defined as $p_{n+1} = t_{n+1} - t_n$.

Bursting period evolution diagrams show, for a burst numbered n, how many iterations have passed since the beginning of burst number $n - 1$. It is worth noting that burst number n of neuron p does not necessarily coincide in time with burst number n of another neuron p'. In fact, it is rarely so. Most of the time they will be at most one cycle apart, but it could well be the case that there is a long time between both.

The time at which a burst occurs equals the sum of all previous periods, i.e., the summation of the area under the curve of the diagram up to the point of that burst.

2.5 Simulating networks of neurons

Neuroscience's aim is to understand the nervous system. In this endeavour it uses tools like electrophysiology to gather data, statistics to analyze those data, mathematics to hypothesize models and computers to test those models and make predictions.

We have created a C++ library that precisely fits in this last area. Both neurons and synapses are usually described by a set of differential equations that, when integrated over time, give as result a set of functions of time representing the neuron's or the synapse's activity. The main goal of this library is to provide some models of neurons and synapses and the methods to integrate them over time.

More specifically, the goals of this library are:

- Reusability: models and mathematical methods must be easily reuseable in other projects, relying for that on C++'s mechanisms.
- Performance: the simulation must be as fast as possible
- Modularity: there must be a clear cut between neurons, synapses, integrators and so on (so that they can be independently reused).
- Flexibility: elements must be easy to combine and connect, and their parameters easy to change.
- Usability: the end user of the library must be able to easily construct networks of neurons.

2.5.1 Advantages of the library

Given the complexity of the CPGs used in this work and the high number of variables they involve, an object-oriented library is a very helpful tool for encapsulating data and behaviour. Using objects helps divide variables and equations that belong together in manageable logical units that can be treated as single entities. The traditional procedural

approach would have all variables and equations in the same scope, which leads to a code harder to change and understand.

The use of C++'s templates makes this library very easy to extend. End users do not need to write much code to add new models. Instead, the library provides extension mechanisms in the form of *wrappers* that allow the user to have a whole family of classes just writing one himself. This reduces redundancy of code (the end user does not need to copy the same code over and over) and favours refactoring, since code is located in one single place.

As a result, it is very easy to implement CPGs comprising different models of neurons and synapses, including simulating them and accessing their parameters and variables. A tutorial on using the library can be found in Appendix A.

2.5.2 Implementation

The library is implemented in C++ for various reasons. The language is natively compiled, which produces very fast executable programs. Furthermore, C++ is a *multi-paradigm* programming language, combining several features of structured and object oriented programming.

A very powerful feature of C++ on which this library heavily relies is parameterization. Template techniques have seen a large development over the last decades, helping in developing highly modular and reusable software.

From the user's point of view, there are three main components in the library:

Models: classes specifying which variables, parameters and equations a given class of systems has.

Integrators: classes that calculate the values of the variables of a given system for some time step.

Wrappers: classes that add some functionality to other classes, much like the decorator pattern [23].

From the developer's point of view there are other two parts, to implement the technique described in [24]:

Concept checking classes: helper classes to check for parametric polymorphism in C++ templates .

Archetypes: classes that implement the basics to meet a concept's requirements.

2.5.3 Wrapping rather than inheriting

The library uses traditional object oriented inheritance combined with parameterization to offer a very flexible functionality extension mechanism: wrappers.

Wrappers' goal is to take away common code from classes implementing the neuron models. By using these wrappers, the end user should be able to combine the extensions he wants, in any arbitrary order. The first idea was to use recurring template inheritance [25]. This technique proved not to be the most effective in terms of how simple model classes could be.

After having written this library, I found out that this technique is called *mix-in programming* in the literature. The key idea is that *mixin*s implement functionality extensions without knowing what is going to be extended (as opposed to plain object oriented inheritance). The user has the freedom to combine this extensions as he wants. For a brief review of the technique, see [26]. Figure 2.9 explains wrapping.

Extension through classical inheritance is bounded to one single extended class, thus extensions must be rewritten for every class to be extended. Wrapper extensions take advantage of parameterization so that one extension is only written once for all extensible classes. It is the compiler who is then responsible for creating the actual derived classes. This approach has, however, certain drawbacks. C++ does not provide a direct method to check if the class to be extended meets certain interface requirements. Siek and Lumsdaine present in [24] a technique called "concept checking" that to some extent solves the problem.

2.5.4 The constructor parameters problem

In C++, all methods are inherited from the base class except for constructors. Therefore the derived class's constructor must explicitly provide the appropriate parameters to construct the superclass. That is, if a class B provides a non default constructor and a class C inherits from B, C will not automatically have B's constructor, but the programmer will have to explicitly implement a call to it in C.

When using wrappers there is no simple way to guess the base class's constructor's parameters, so a generic constructor cannot be implemented. This is known as the *mixin* constructor problem, illustrated in Figure 2.10.

In this library the problem is partially solved using a ConstructorArguments class. There is a wrapper, the class SystemWrapper, that creates one such class for the model it is wrapping. This class is then exported in every wrapper as a nested typedef, so successive wrappers can be applied to the same model, and all of them will be able to provide the necessary arguments to the SystemWrapper's constructor.

In [27], many different partial solutions are reviewed and a complete solution is proposed. Future versions of the library will adapt this solution and provide more flexible wrapping mechanisms.

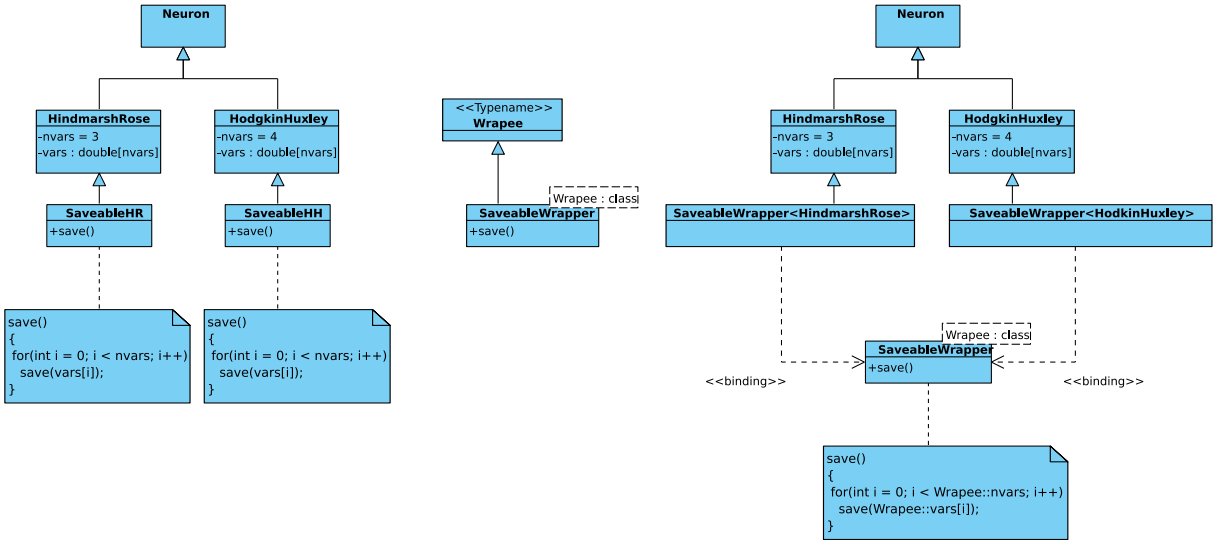


Figure 2.9: *Difference between classical inheritance and extension through wrappers. We have two different classes, HindmarshRose and HodgkinHuxley implementing two different models, one with three variables and one with four variables. We want to add a generic feature: saving variables to a file. This cannot be implemented in Neuron, since the variables to be stored are defined in classes lower in the hierarchy. Using classical inheritance, the only way to achieve it is to duplicate the code in two separate classes: SaveableHR and SaveableHH. Using the pattern shown in the middle of the figure, a parameterized class that derives from the class supplied as parameter, the extension of saving variables needs only be written once. As shown to the right, the derived classes that can save the state of a HindmarshRose or a HodgkinHuxley model are created just by passing the corresponding class as the template argument to the wrapper, no new code is needed.*

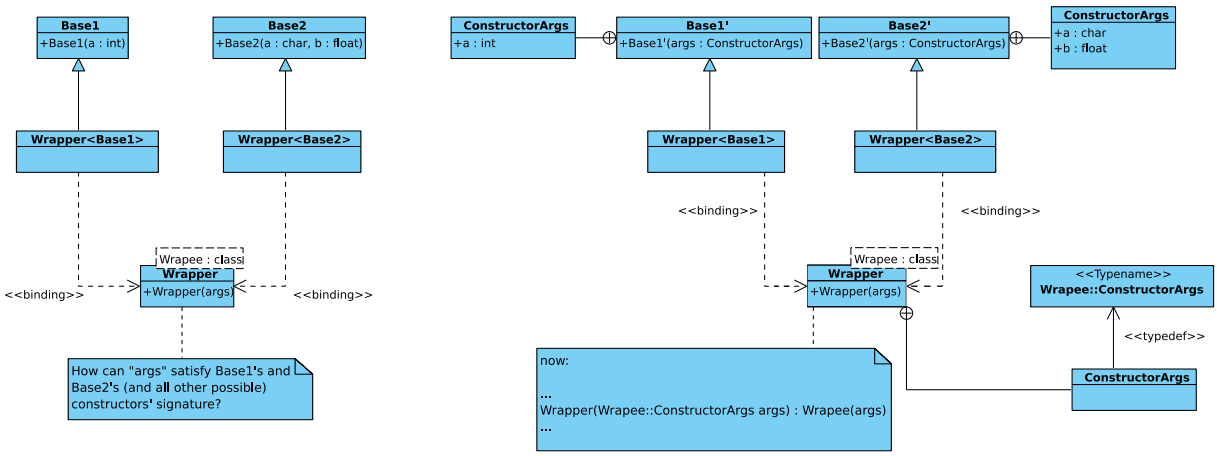


Figure 2.10: *Derived classes must explicitly provide the parameters for the constructors of their base classes. A problem arises if the signature of the base class's constructor is not known, as is the case with wrappers. What signature should the wrapper's constructor have to be generic enough? This is known as the "constructor parameters problem". The approach taken in this library is that every model class must typedef a ConstructorArgs class, that every wrapper will export. This way all constructors will have the same signature, although ConstructorArgs will be a different type for each case.*

3

Central Pattern Generators for Intermodule coordination

Now that we have modules capable of oscillating in isolation, we are interested in coupling them so that a travelling wave emerges that makes the robot displace itself in a coordinated fashion.[1]

3.1 Uncoupled modules

3.1.1 Motivation

In order to see what dynamical features the different proposed CPG models will have, it is necessary to have an idea about how the system works when modules are uncoupled.

The expected behaviour is that all modules will oscillate at the same frequency with a constant phase difference, whose value depends only on the initial conditions. Figures 3.1 and 3.2 show that this is only partly true. Periods are very stable, although not constant, and phase difference is not constant at all. Its drift is, however, constant (save for the initial transient period). This shows that the actual frequency of oscillation of one module depends on the phase difference between the promotor and the remotor neuron within the module and the particular initial conditions of each one.

Although surprising, we can take advantage of this intermodule phase drift to study the proposed central pattern generating circuits: if intermodule phase difference is different from constantly drifting, the proposed connectivity actually has an effect on phase difference.

[1] In the following sections, diagrams will only show the neurons involved in generation and coordination of the rhythm. Motor neurons and any other mechanical parts will be left out.

In every case each promotor or remotor neuron will correspond to only one module, even though no explicit division will be made, to maintain diagrams clean.

Unless otherwise stated, all models are symmetric, meaning that the same scheme for the remotor neurons also applies to the promotor side, and it will be omitted when the resulting diagram is too unwieldy. Remotor and promotor neurons within one module will keep the organization of Figure 2.7(a).

3.1.2 Parameters

- The parameters used for the Rulkov model in all neurons are:
  - $\alpha = 15$, $\mu = 0.001$, $\sigma = -0.33$, $\beta = 0$ and $\sigma_e = 1$.
- Parameters for contralateral synapses:
  - $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = 9$, $g_{syn} = 1.5$, $T = 1$, $release_time = 0.01$.

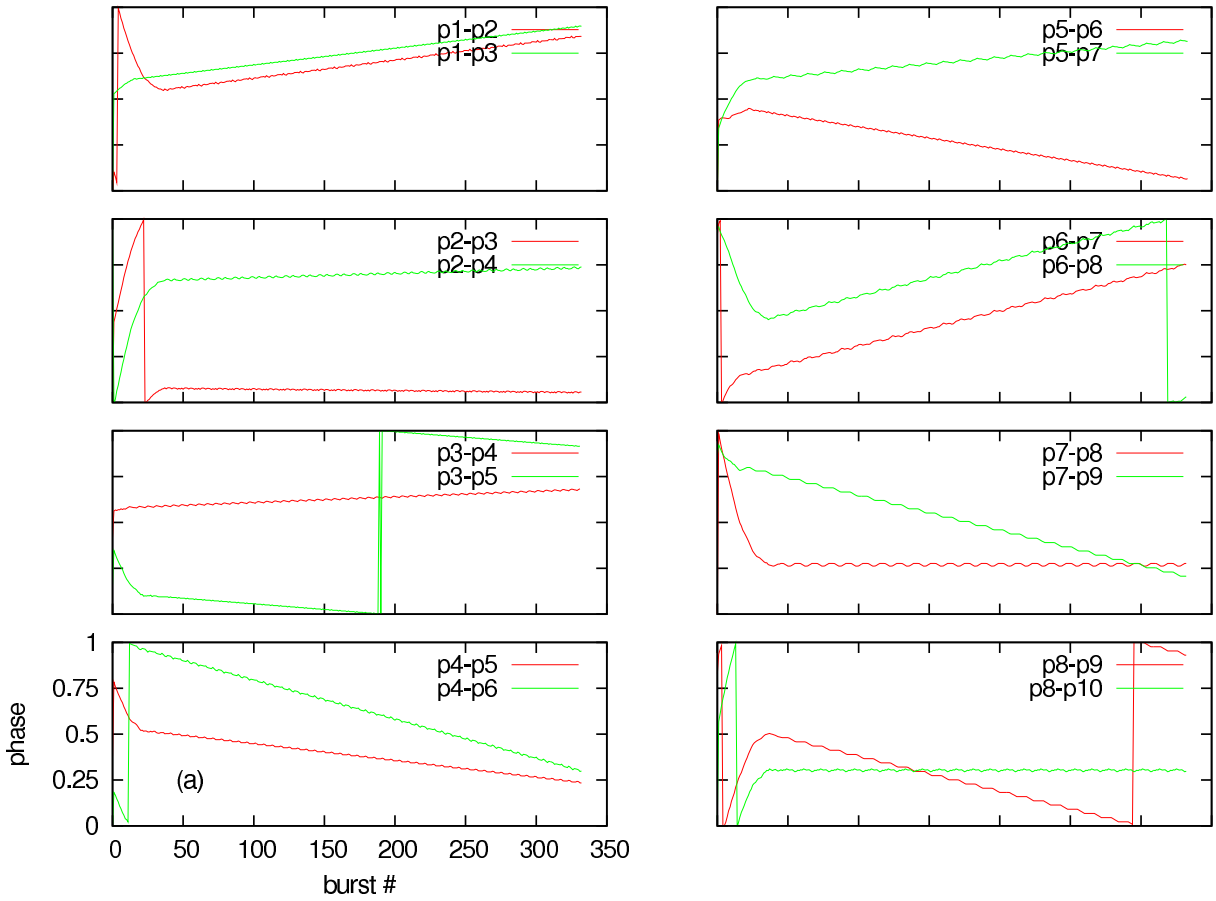


Figure 3.1: *Uncoupled modules: phase differences for independently oscillating modules. There are two differentiated regimes, see for example the p8-p9 curve. There is a steep rising of the curve and then, at approximately burst #50 the curve steadily decays in a much smoother way. The first regime is due not only to frequency difference, but to a change in frequency. Both modules 8 and 9 are going through an adjustment between their promotor and remotor neurons. After this adjustment frequency settles down differently for each module, and is the only source of (constant) phase drift in the second regime.*

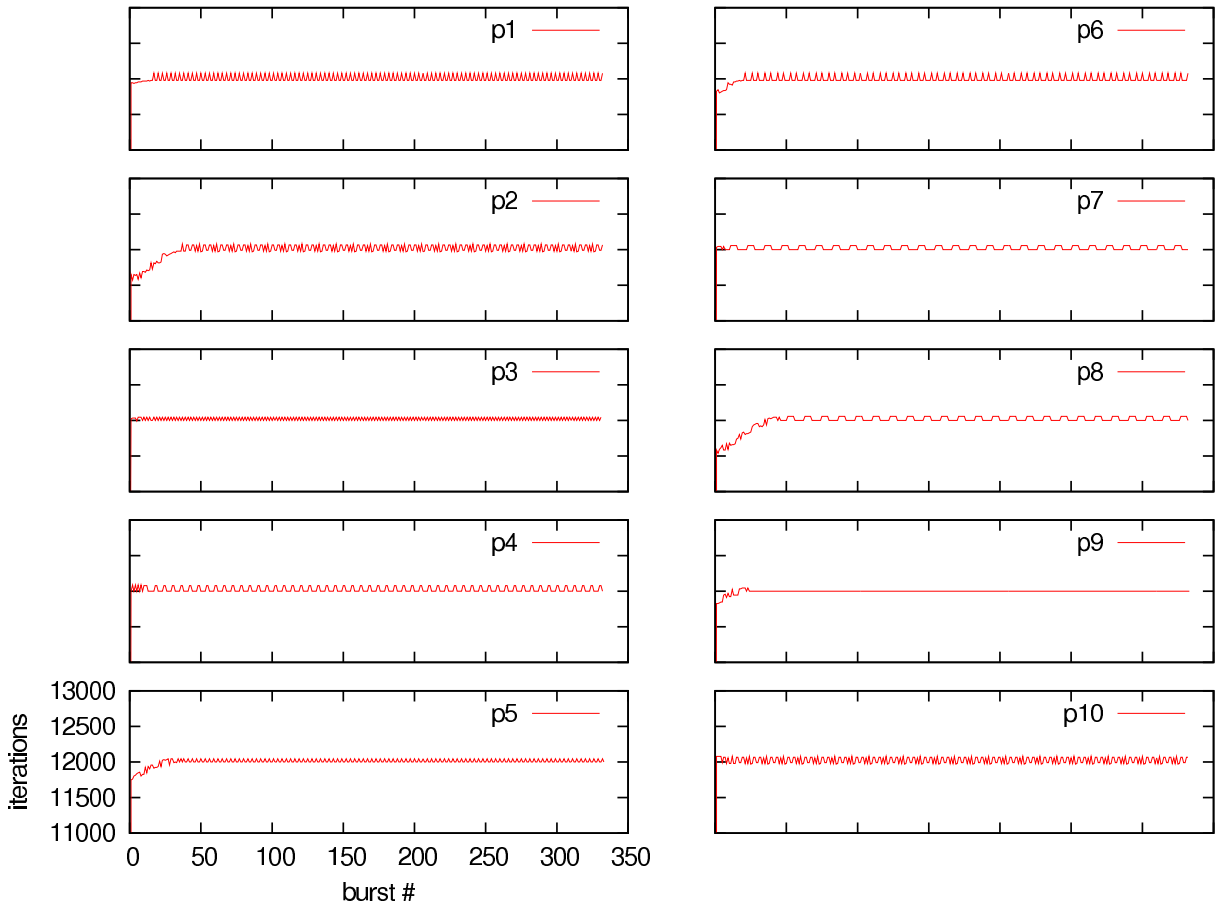


Figure 3.2: *Uncoupled modules: periods for independently oscillating modules. Modules 2, 5, 6, 8 and 9 show a period of adjustment at the beginning due to a transient synchronization between their promotor and remotor neurons. This synchronizations carry along a decrease in frequency (increase in period) of the respective neurons, as cleraly seen in p2 and p8.*

3.2 Symmetrical inhibition, weak coupling

3.2.1 Motivation

The first attempt was to connect adjacent modules with the same scheme as the alternating neurons within a module, that is, with two inhibitory synapses, one in each direction (two for promotors and two for remotors). These synapses have a low conductance so that their activity would influence the postsynaptic neuron's frequency but would not inhibit them completely. The idea is that inhibitory synapses should prevent adjacent neurons from firing synchronously, thus giving rise to a certain firing sequence. The question is whether this sequence would be sufficient to make the robot move in an effective manner.

3.2.2 Diagram

The scheme is as shown in Figure 3.3

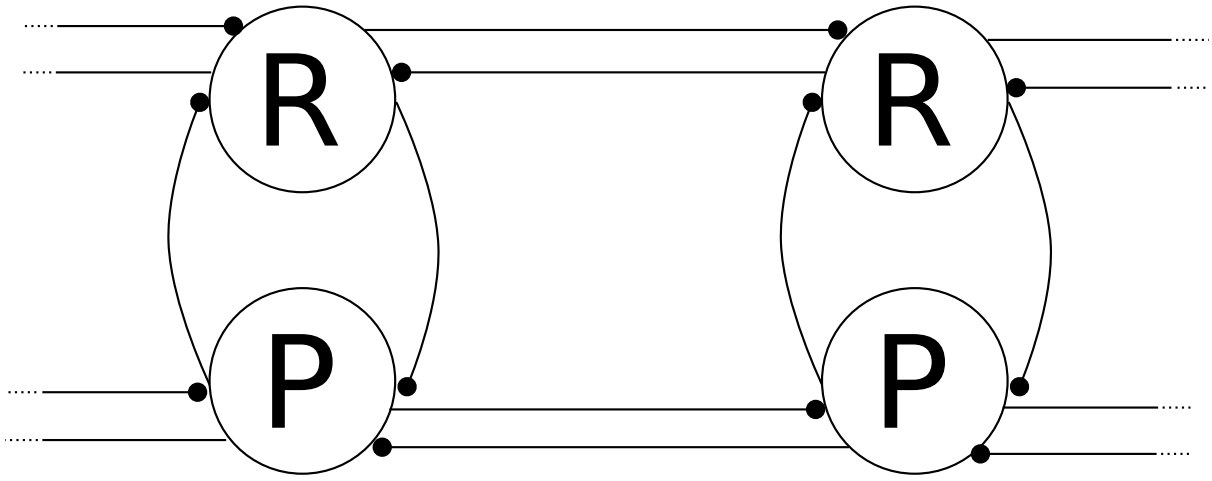


Figure 3.3: *Two adjacent modules connected with the symmetrical inhibition model.*

3.2.3 Parameters

- The parameters used for the Rulkov model in all neurons are:
  - $\alpha = 15$, $\mu = 0.001$, $\sigma = -0.33$, $\beta = 0$ and $\sigma_e = 1$.
- Contralateral synapses have these:
  - $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = 9$, $g_{syn} = 1.5$, $T = 1$, $release_time = 0.01$.
- Ipsilateral synapses have the same parameters except for $g = 0.2$.

3.2.4 Evaluation

In this particular example, the model has achieved a stable state with constant phase difference towards the end of the simulation, but this does not always happen. It remains for further investigation under which initial conditions the system arrives at a stable rhythm.

This CPG works by making small frequency adjustments. Synapses going from one module to another are weak, meaning that the effect that they have on the receptor is only small (the receptor's activity is not completely inhibited, but rather only accelerated or slowed down). As Figures 3.4 and 3.5 show, changes in period and phase difference are very smooth, but very slow too.

Although no single synapse goes further than the first neighbouring module, adjustments have a global effect. Period evolution in Figure 3.5 shows a strong correlation among plots. For instance, all plots show a local minimum around burst #50.

By letting some modules oscillate faster than their neighbours, it is possible to switch the sequence in which they activate. We call this phenomenon *sequence inversion*. It is reflected as a crossing of the curves in one subplot of Figure 3.4 (label (a) in the figure). Sequence inversion means that, if three consecutive modules display a propagating wave in one direction, it will suddenly start propagating in the opposite direction. The figure shows that this sequence inversion happens locally (inversion happening in three consecutive modules does not imply its happening in other three different modules).

It has not been studied under which initial conditions sequence inversion happens. Furthermore, if this phenomenon can take place in any three consecutive modules, we have no a priori control of the direction of movement of the robot (appart from setting specific initial conditions).

Advantage could be taken of this sequence inverting phenomenon if we could determine what makes it happen. Then, we could invert the direction of displacement without changing the underlying circuit.

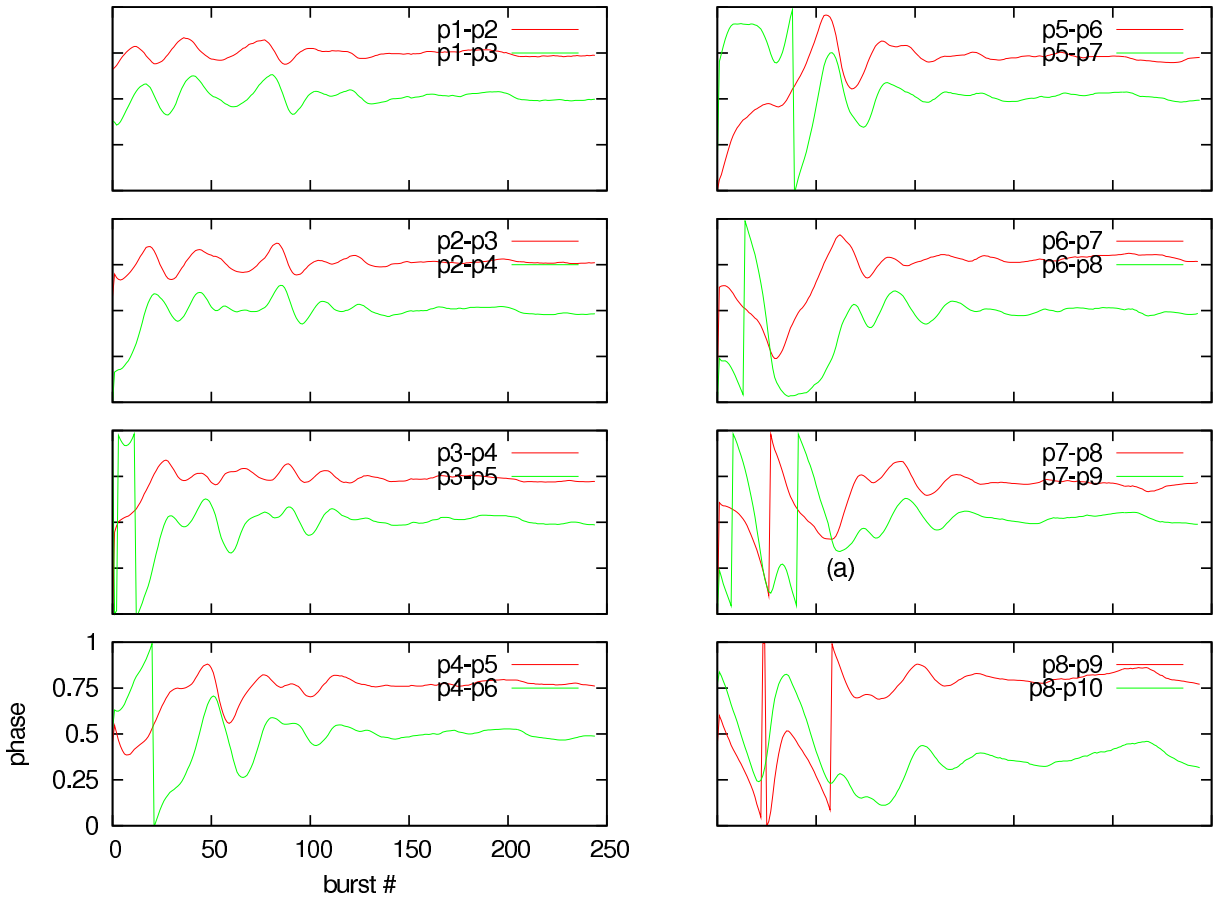


Figure 3.4: *Phase differences starting with random initial conditions for the symmetrical inhibition model, weak coupling ($g = 0.2$). This CPG works by making small frequency adjustments. Phase difference changes smoothly until a stable state has been found. (a) This crossing of the phase curves represents a sequence inversion: module 9 is allowed to overtake module 8 in order to correct the direction of propagation of the oscillation.*

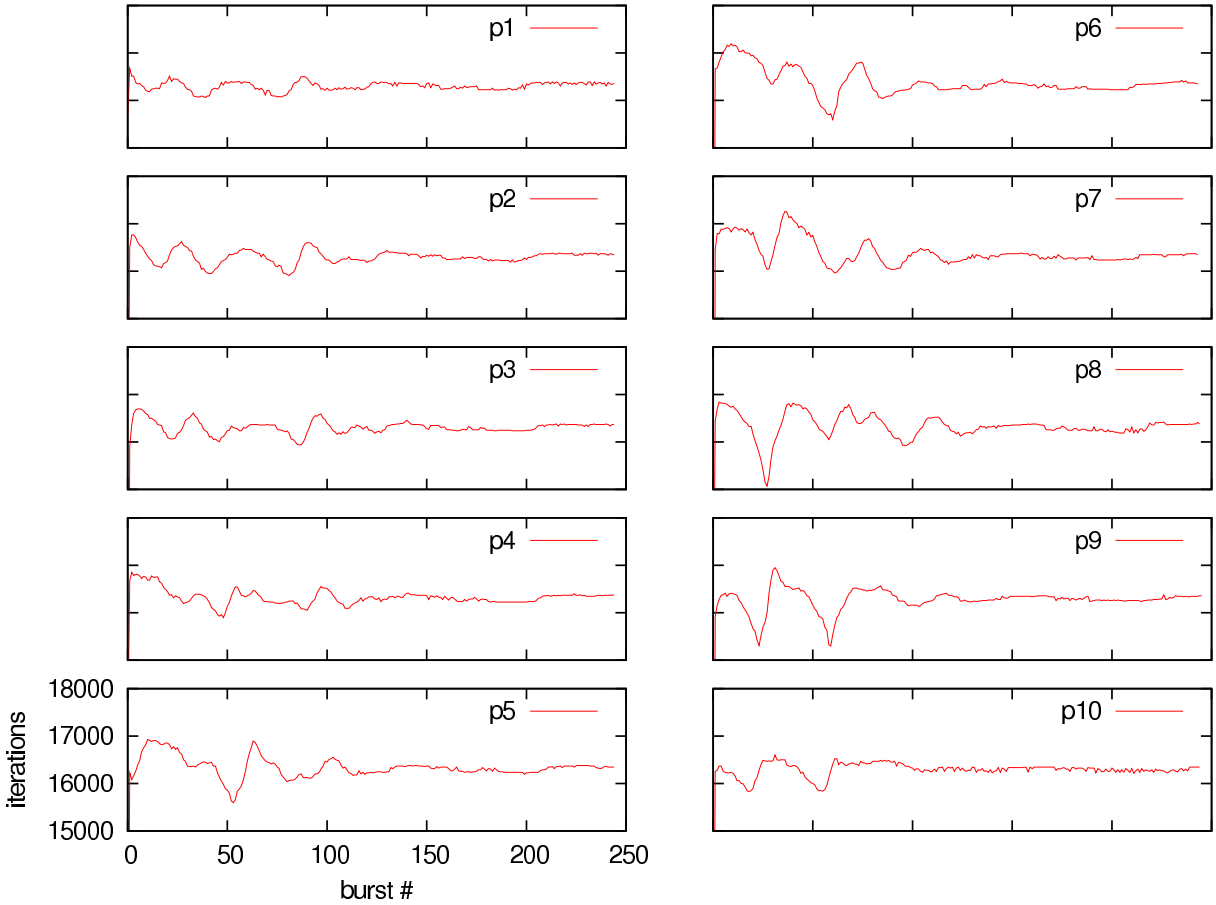


Figure 3.5: *Periods starting with random initial conditions for the symmetrical inhibition model, weak coupling ($g = 0.2$). There is a strong correlation among plots: all of them show a local minimum around burst #50. This indicates that local adjustments are propagated at a global scale, eventhough no single synapse projects further than first neighbour modules.*

3.3 Symmetrical inhibition with strong coupling

For this CPG we take the previous model and strengthen ipsilateral conductance to $g = 1.5$ (the same as contralateral), so that the receptor's activity is completely inhibited. Behaviour will now depend on the recovery time of a neuron between succesive inhibitions. We describe three kinds of activity: irregular, parity synchronization mode 1 and parity synchronization mode 2.

3.3.1 Irregular activity

If neuron parameters are kept as in the previous CPG but synaptic conductances are strengthened, one neuron may *dominate* its neighbours for a number of periods and then suddenly lose control. Figure 3.6 shows a sample of this irregular activity.

In this case, one neuron can be completely inhibited if its two neighbours burst simultaneously, as is the case with neuron p5 in Figure 3.6. Although it cannot completely recover between successive inhibitions, it can reach an ever higher potential, finally leading it to release. This neuron can now inhibit its neighbours.

In Figure 3.7 it can be observed how neurons vary their periods of activity. There is no apparent pattern that indicates periodicity, but we have not carried out any tests about the chaoticity of the system either.

3.3.2 Parity synchronization mode 1

This mode arises when neurons do not have a chance to recover from inhibition and they cannot achieve a higher potential each time. This way, once a neuron is inhibited, it stays that way forever.

If we take three consecutive neurons: p3, p4 and p5, for instance, it might happen that p3 inhibits p4, thus releasing p5 from inhibition. Now both p3 and p5 are free from p4's inhibition and if they fire, they will inhibit, in turn, p2 and p6. If they manage to synchronize in anti-phase, it might happen that p4 never gets a chance to recover due to the total effect of alternate inhibition. This phenomenon can spread across the whole length of the system, resulting in a synchronization of all the even numbered neurons.

This kind of *parity synchronization* is found when using the following parameters:

- Neurons:
  - $\alpha = 20$, $\mu = 0.001$, $\sigma = -1.33$, $\beta = 0$ and $\sigma_e = 1$.
- Synapses (contralateral and ipsilateral):
  - $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = 13$, $g_{syn} = 1.5$, $T = 1$, $release_time = 0.01$.

Figure 3.8 shows an activity sample of this kind of synchronization. As Figure 3.10 reflects, almost all even neurons maintain a constant phase difference. However, Figure 3.9 reveals a strange behaviour between

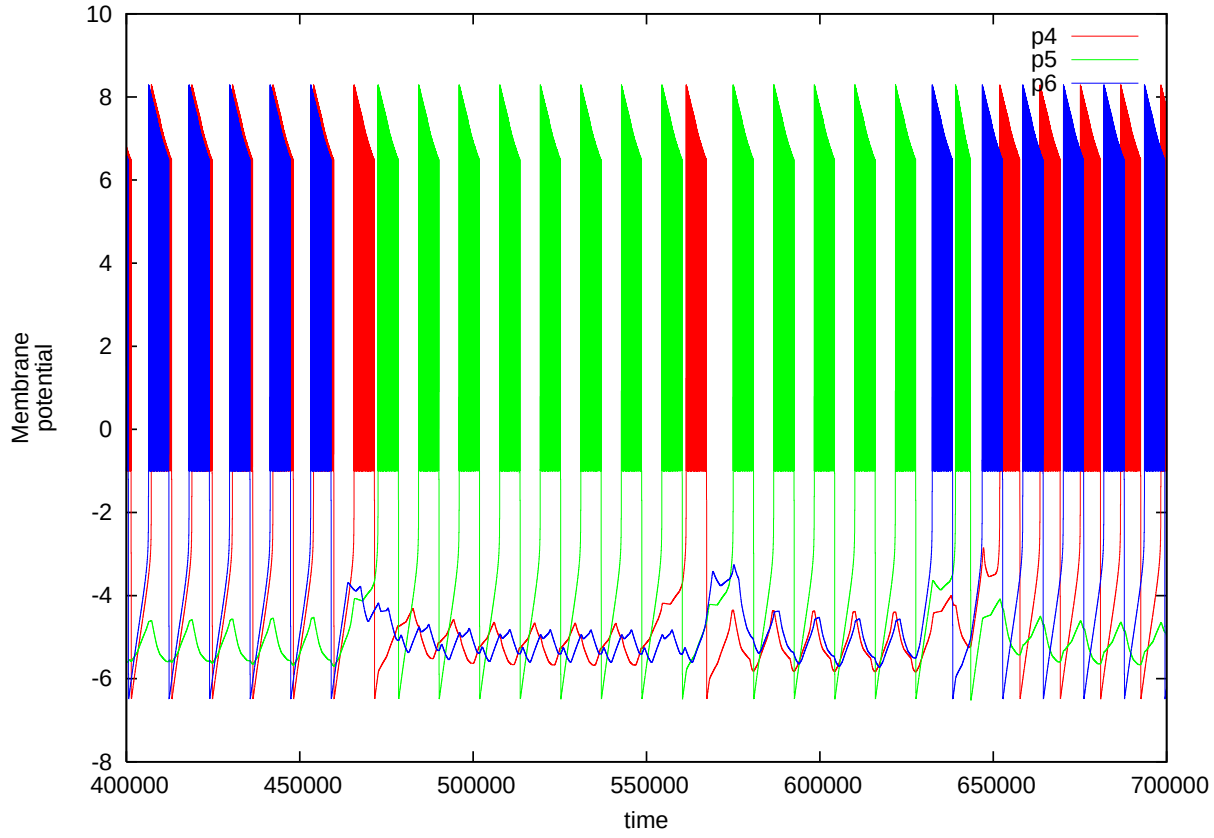


Figure 3.6: *Irregular activity under symmetric inhibition, strong coupling ($g = 1.5$). Neurons can completely inhibit their neighbours. In this case there is not enough time to completely recover between successive inhibitions, but recovery speed is enough for a neuron to reach ever higher potentials. This will eventually lead the neuron to burst, thus possibly inhibiting its neighbours.*

3.3.3 Parity synchronization mode 2

If the interburst time is increased so that inhibited neurons have time to fully recover, another kind of *parity synchronization* emerges. Now, the middle neuron, p4 for instance, has time to recover from both inhibitions from p3 and p5 and, in turn, inhibits them simultaneously. After this, p3 and p5 also recover simultaneously, inhibiting p4. This time second neighbours are synchronized in phase, and in anti-phase with first neighbours.

This different kind of *parity synchronization* is found when using the following parameters:

- Neurons:
  - $\alpha = 6$, $\mu = 0.001$, $\sigma = -0.33$, $\beta = 0$ and $\sigma_e = 1$.
- Synapses (contralateral and ipsilateral):

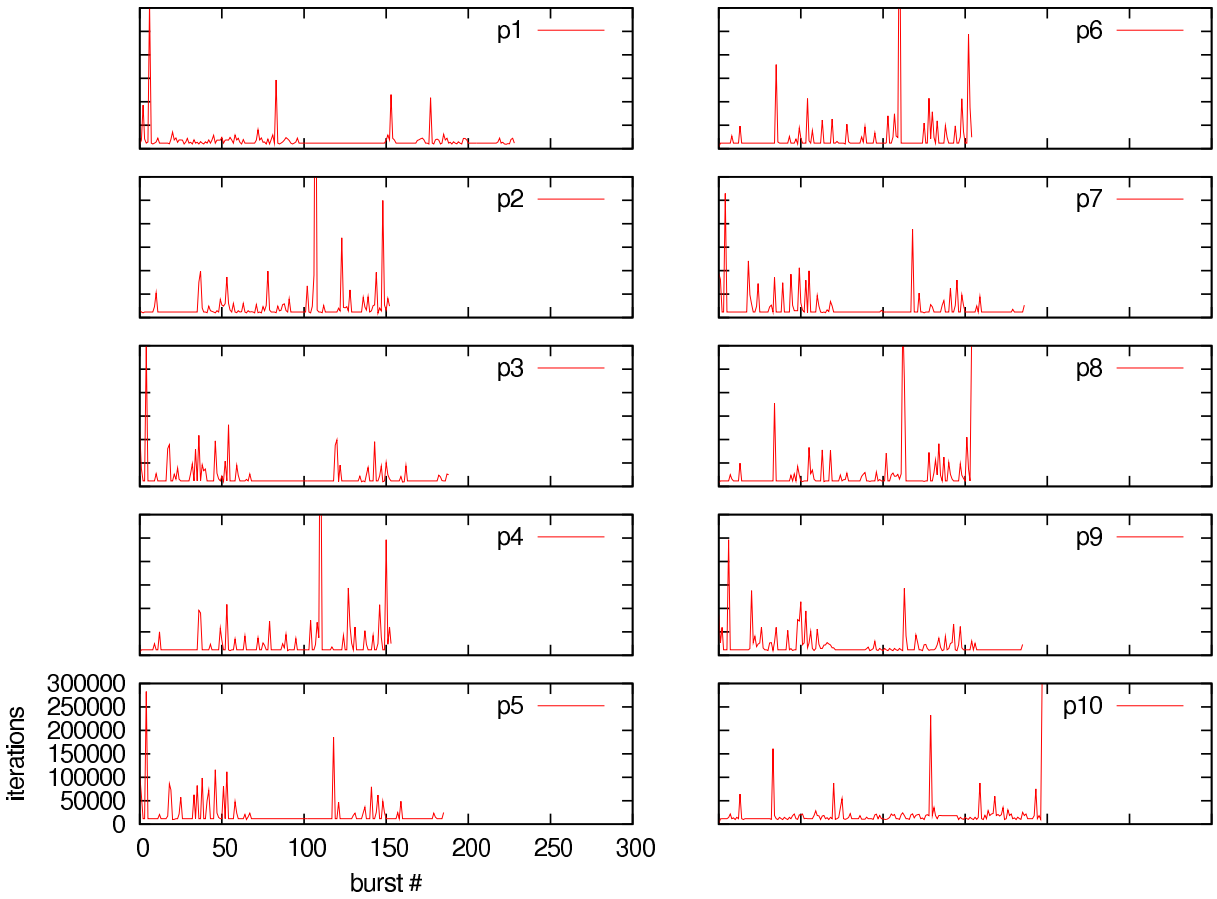


Figure 3.7: *Periods of bursting under symmetric inhibition, strong coupling, irregular activity. All neurons show long times of inactivity (large peaks) followed by regular intervals of activity (iterations between bursts below 50000). There is no apparent sign of periodicity but no tests have been carried out about the chaoticity of the system.*

– $\alpha = 0.5, \beta = 10$, $\tau = 0$, $E_{syn} = 3$, $g_{syn} = 1.5$, $T = 1$, $release_time = 0.01$.

3.3.4 Evaluation

Symmetrical inhibition with strong coupling is not a good candidate for our purpose. The irregular activity mode is clearly not fit for a stable controller. The first mode of parity synchronization completely disables control for half of the modules, and the second parity synchronization mode produces a standing wave rather than a propagating wave across the body of the robot.

We can extract one conclussion from the last mode of operation: phase difference between adjacent modules must be different from 0.5, otherwise a standing wave will appear rather than a propagating wave.

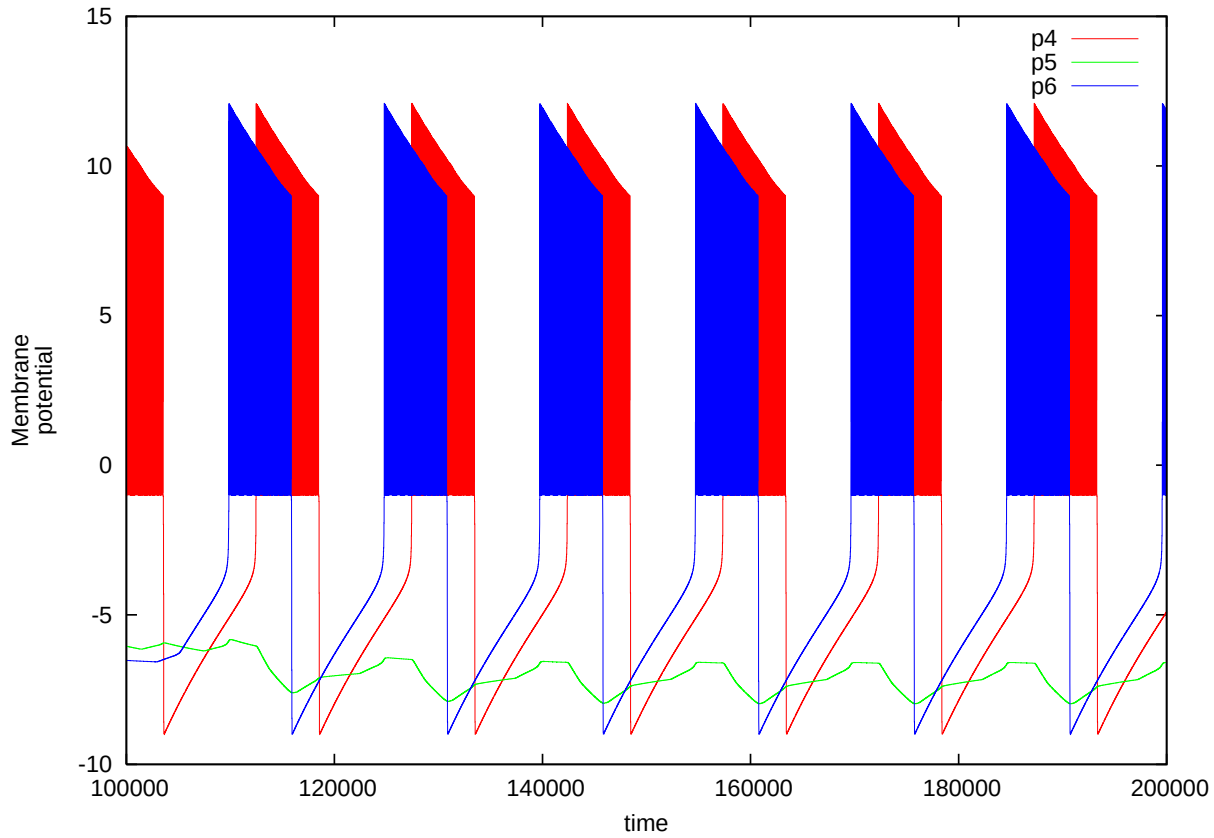


Figure 3.8: *A sample of parity synchronization mode 1: odd numbered neurons oscillate approximately in anti-phase completely inhibiting even numbered neurons.*

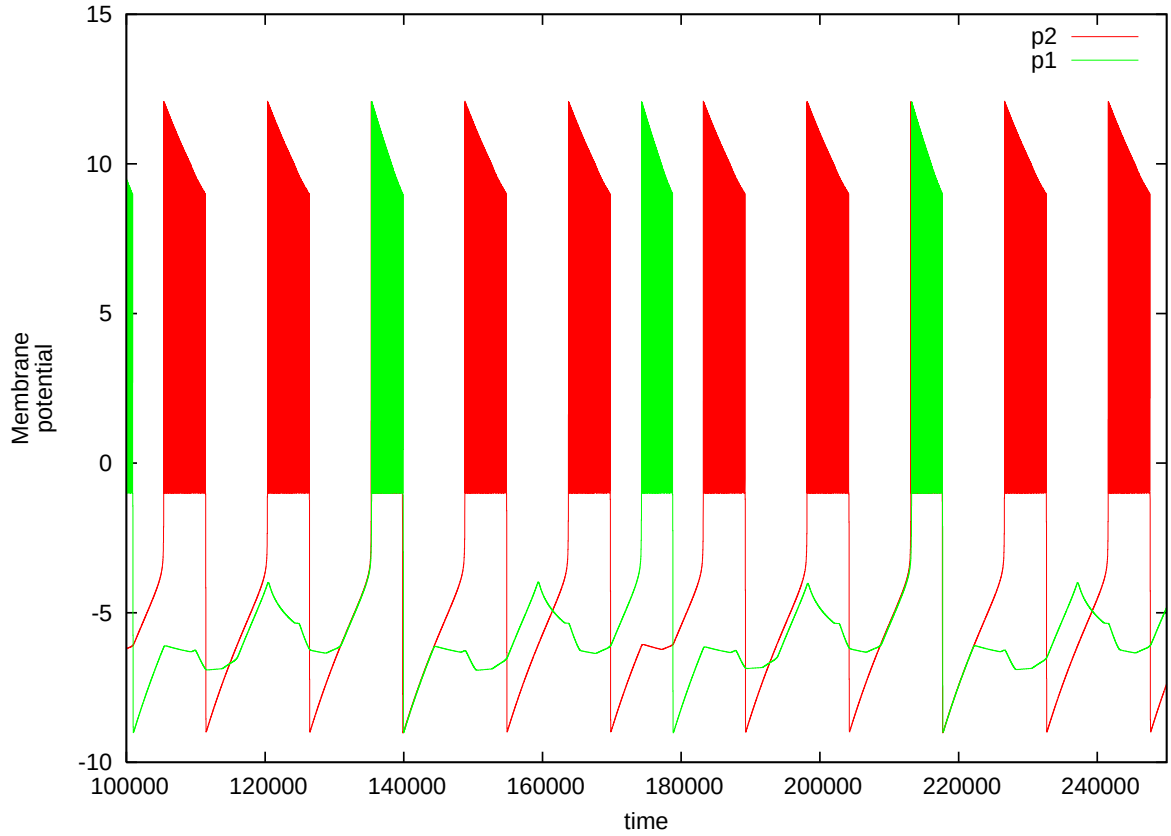


Figure 3.9: *Parity mode 1: 3:6 frequency locking between neurons p1 and p2, i.e., p1 bursts three times for every six bursts of p2. This is a result of p1 being a border neuron and receiving inhibition only from p2.*

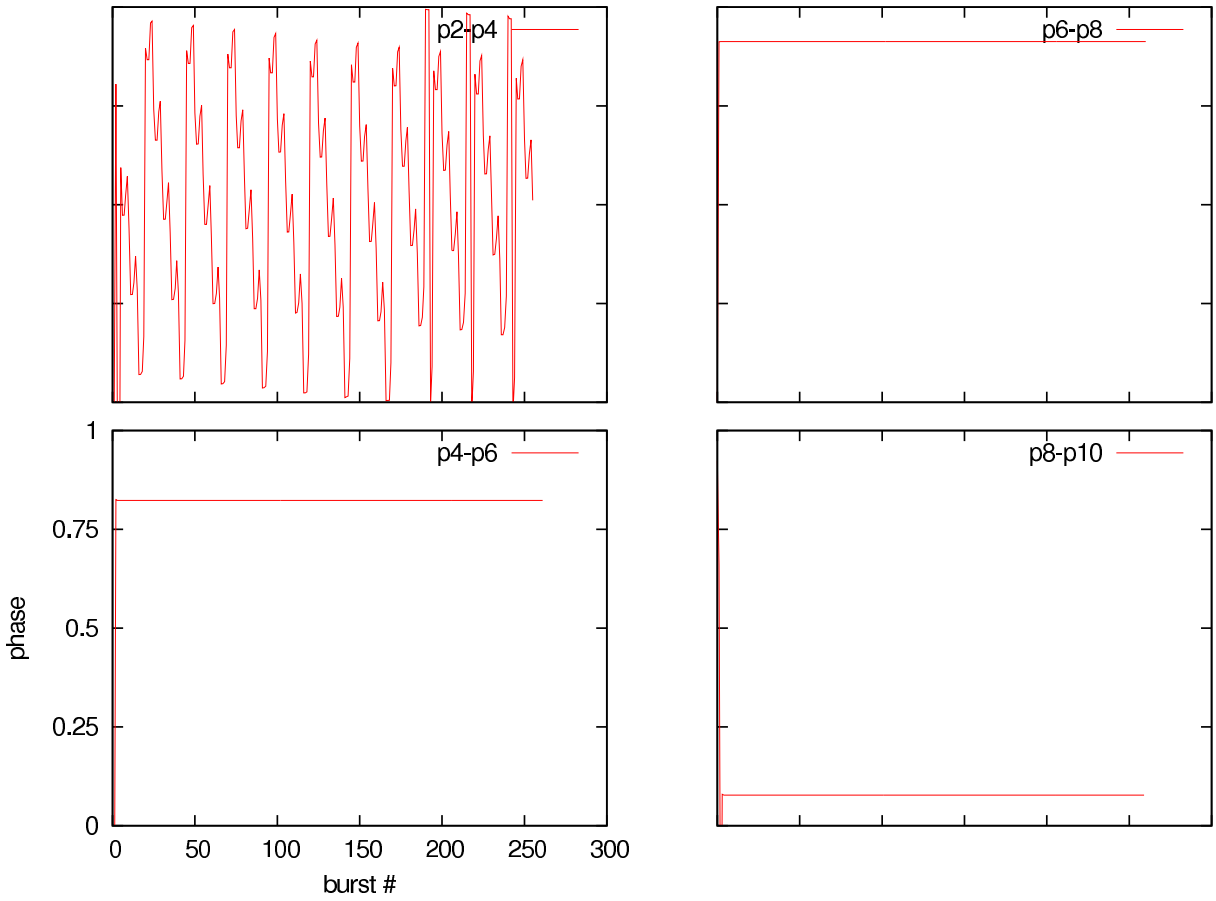


Figure 3.10: *Phase differences of bursting neurons in parity mode 1: odd numbered neurons are completely inhibited by even numbered neurons (neurons not shown remain under threshold and do not burst). The activation of these neurons in different phases inhibits all other neurons completely. In this particular case, p2 does not completely inhibit p1 and it gets a chance to fire every three bursts of p2. This 3:6 frequency locking causes the irregularities seen in p2-p4.*

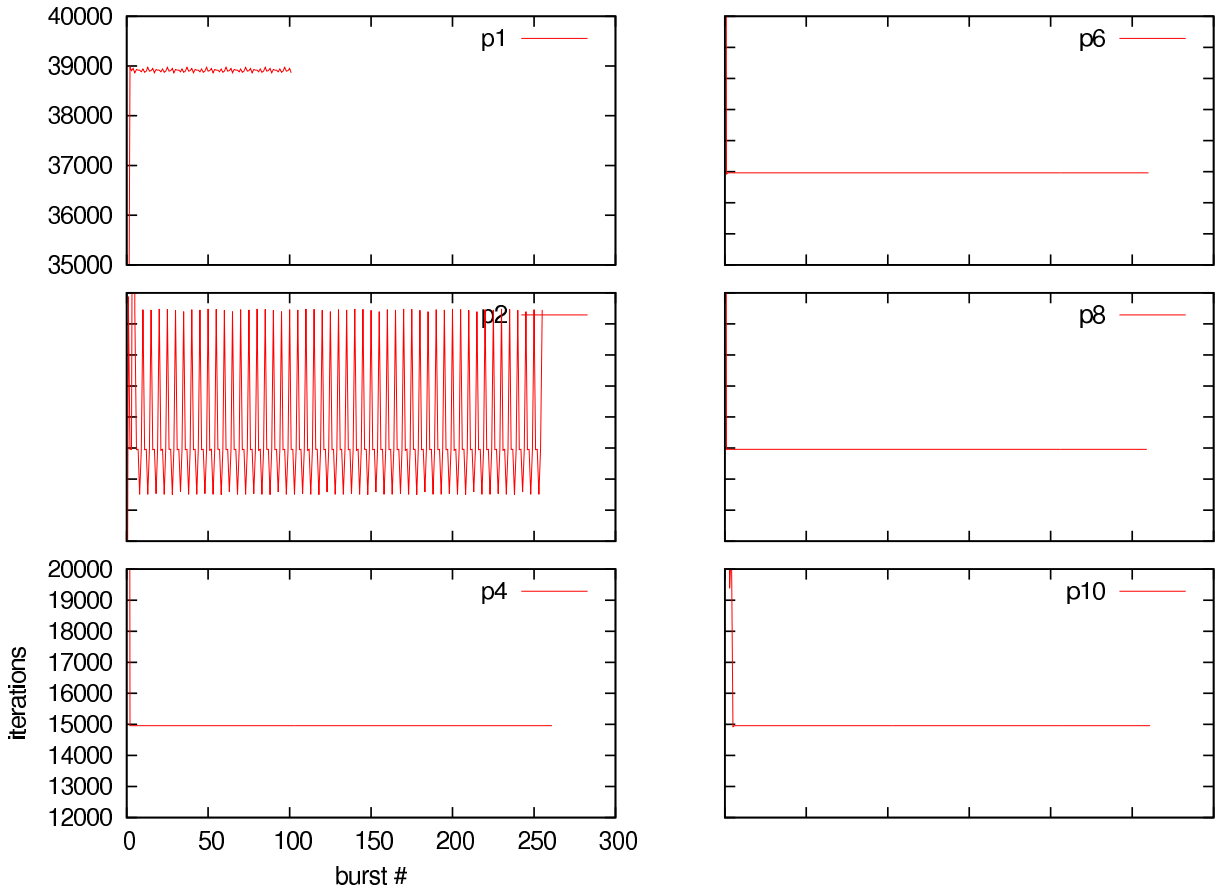


Figure 3.11: *Periods of bursting neurons in parity mode 1 (unlabeled axes share the scale of the lower left plot): odd numbered neurons are completely inhibited by even numbered neurons, neurons not shown remain under threshold and do not burst. P1 is an exception, it bursts three times for every six bursts of neuron p2. Irregularities in p2 are caused by p1 bursting with a much lower frequency, which delays one in six bursts of p2.*

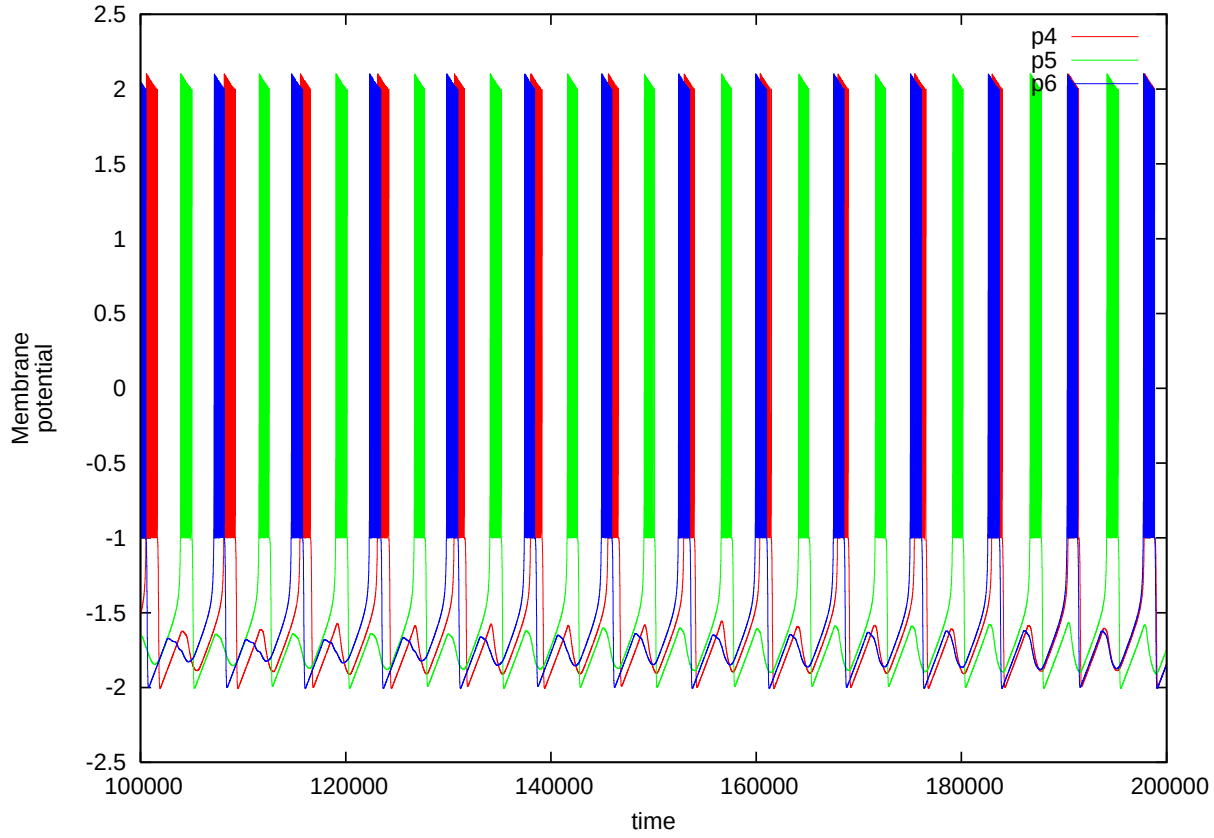


Figure 3.12: *A sample of parity synchronization mode 2: neurons have time to fully recover from inhibition. In-phase synchronization with second neighbours, anti-phase synchronization with first neighbours.*

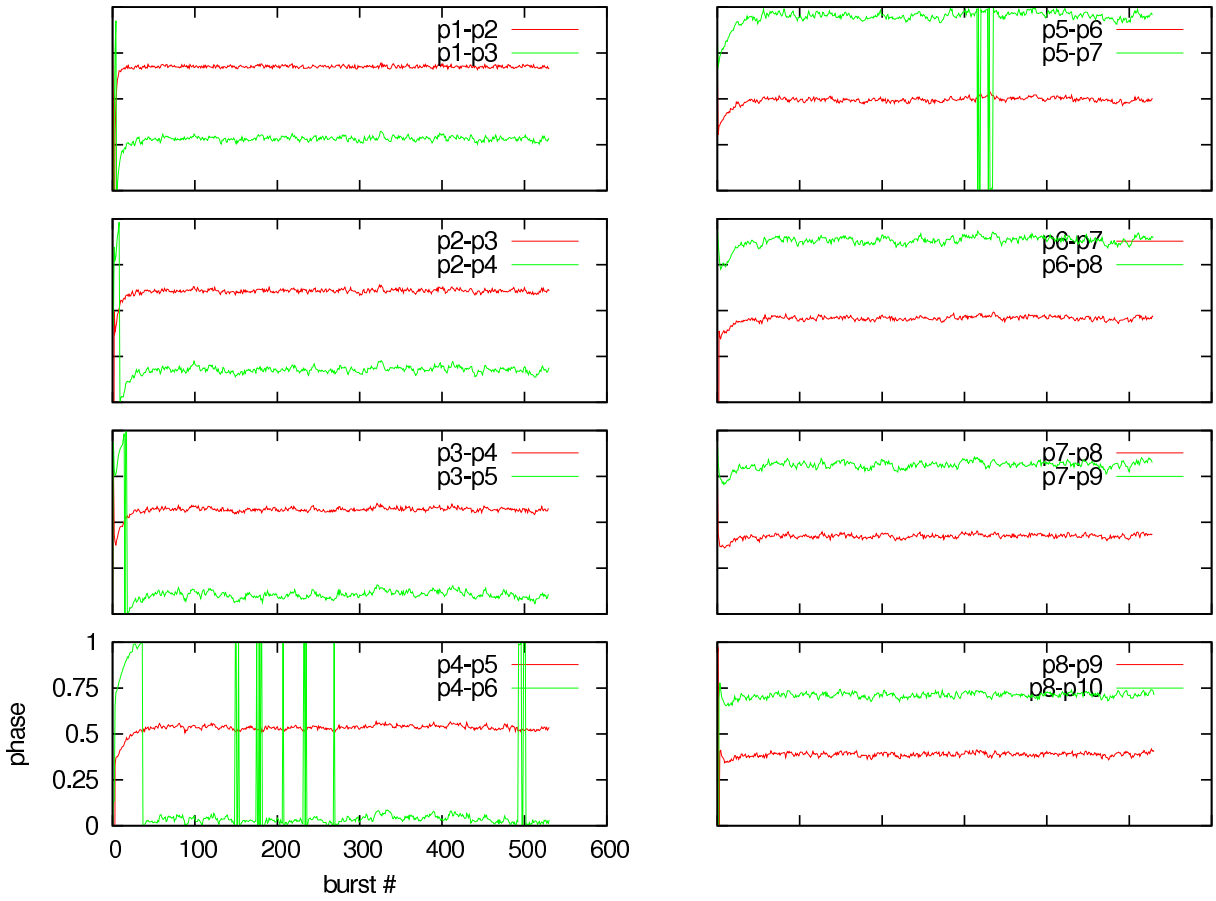


Figure 3.13: *Phase differences of bursting neurons in parity synchronization mode 2: in-phase synchronization with second neighbours, anti-phase synchronization with first neighbours. (note how, although stable, phase difference is not constant across modules). Phase difference to first neighbours is approximately* $1/2$*. This prevents a travelling wave in the body of the robot from emerging.*

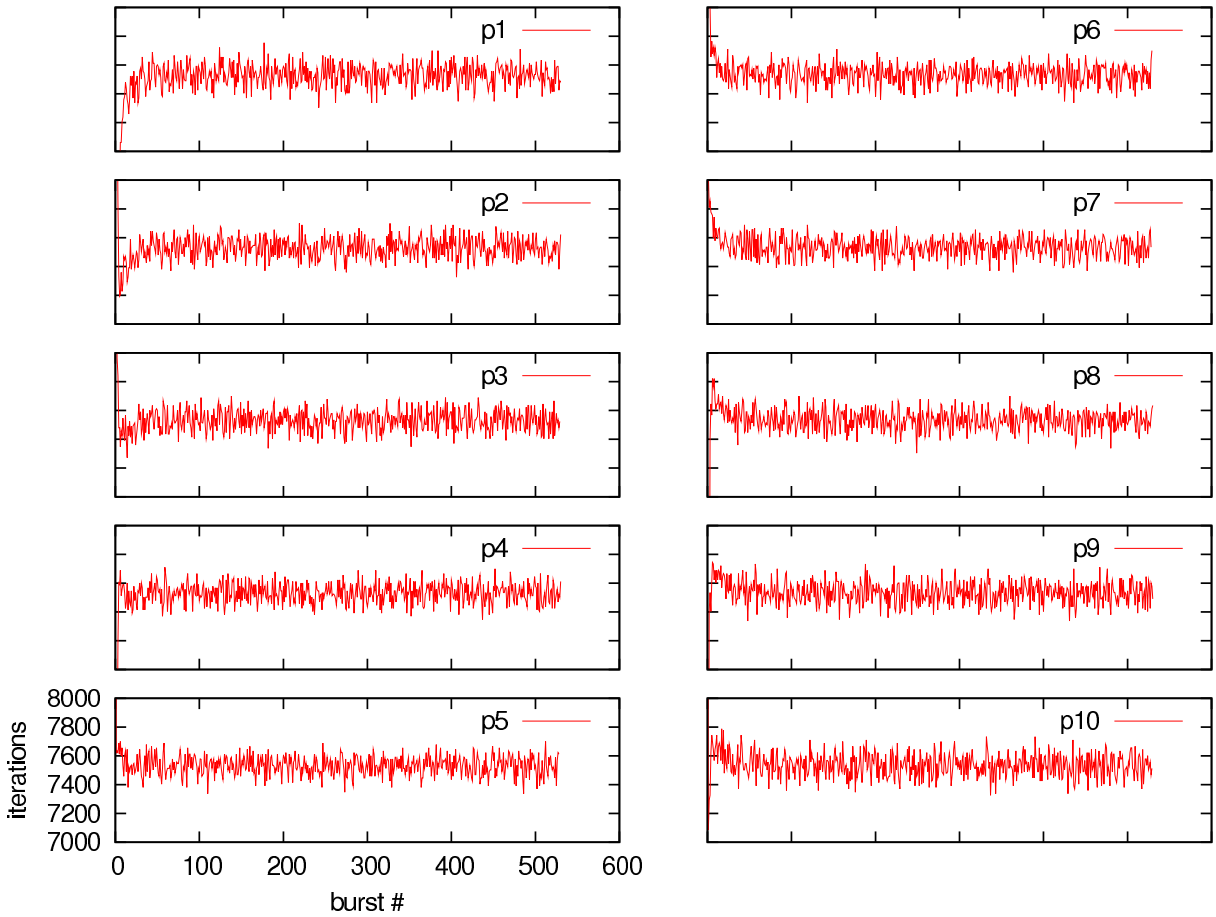


Figure 3.14: *Periods of bursting neurons in parity mode 2: in-phase synchronization with second neighbours, anti-phase synchronization with first neighbours. Period does not provide much information now, since phase difference between modules has already discarded this CPG as a good candidate.*

3.4 Inhibitory loop

3.4.1 Motivation

It is clear that the previous symmetric approaches are not in the right direction. Weak coupling lacks a mechanism that imposes a travelling direction and strong coupling is not able to generate a travelling wave at all.

This scheme is the first asymmetric connectivity proposed. The goal is to achieve a constant delay between adjacent modules by adding a specific circuit that creates a constant phase difference.

As Figure 3.15 shows, there is an intermediate neuron labeled "side" between every two promotor and every two remotor neurons. This neuron is in a silence regime except when excited, in which case it enters a tonic spiking regime. When the excitation ceases, "side" neurons will return to their silent state.

3.4.2 Parameters

- Promotor and Remotor neurons:
  - $\alpha = 15$, $\mu = 0.001$, $\sigma = -0.33$, $\beta = 0$ and $\sigma_e = 1$.
- "Side" neurons (silent when idle, tonic spiking when excited):
  - $\alpha = 3$, $\mu = 0.001$, $\sigma = 0.25$, $\beta = 0$ and $\sigma_e = 1$.
- Contralateral synapses:
  - $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = 13$, $g_{syn} = 1.5$, $T = 1$, $release_time = 0.01$.
- Excitatory synapse to neighbour neuron:
  - $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = -7$, $g_{syn} = 3$, $T = 1$, $release_time = 0.01$.
- Excitatory synapse to "side" neuron:
  - $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = -2$, $g_{syn} = 5$, $T = 1$, $release_time = 0.01$.
- Inhibitory synapse to "side" neuron:
  - $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = 1.1$, $g_{syn} = 5$, $T = 1$, $release_time = 0.01$.
- Inhibitory synapse from "side" neuron to motor neuron:
  - $\alpha = 0.5$, $\beta = 10$, $\tau = 0$, $E_{syn} = 9$, $g_{syn} = 5$, $T = 1$, $release_time = 0.01$.

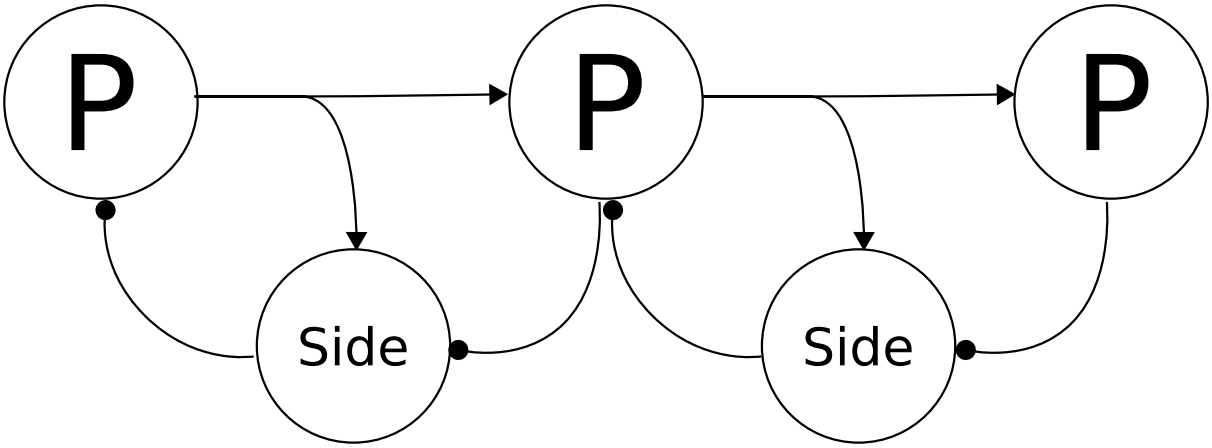


Figure 3.15: *Inhibitory loop (remotors side symmetric, ommited for clarity).*

3.4.3 Evaluation

The system works as follows. Let's take only one P neuron and its "side" neuron (the one it excites and receives inhibition from). The P neuron excites the "side" neuron when it (P) bursts. Under excitation, the "side" neuron enters a tonic spiking regime, which activates the inhibitory synapse to P. This inhibition is weak, so it does not completely inhibit P, but it reduces its frequency. The "side" neuron will remain in the tonic spiking regime for some time, even past P's burst, until it eventually returns to the silent regime. At a steady state, P's frequency under "side"'s influence is lower than its nominal frequency.

We now introduce one neighbour P neuron, let's call it P', to which P excites and who, in turn, inhibits "side". P' at its nominal frequency oscillates faster than P with "side"'s influence. If P' receives excitation it will oscillate even faster. Furthermore, at equal frequency of oscillation, excitation will tend to synchronize both P and P' in phase.

Let's see how this subsystem with two promotor neurons and one side neuron works. P' is oscillating faster than P, so it will slowly reduce its phase difference with the latter. By reducing phase difference it inhibits "side" earlier every cycle, so its spiking episodes become shorter. With less inhibition in each cycle, P begins to oscillate faster.

There is a given phase difference between P and P' that is a stable state. That means that both P and P' oscillate at the same frequency, which seems surprising given that P only receives inhibition. The fact is that at this point, inhibition from side only affects burst length, making it shorter, but does not affect P's recovery process, thus effectively making frequency higher than its nominal frequency. Here, "side" acts as a phase detector.

From the phase differences (Figure 3.16) we can tell that this model achieves a very stable rhythm with stable phase differences between all modules. Burst periods are relatively regular, with border modules having less variation than middle ones.

To investigate the effect of the loop, we first suppress it and leave only the one-way excitatory path. Figures 3.18 and 3.19 clearly show that phase maintenance can be achieved with only the excitatory path.

It is known that excitatory activity promotes in-phase synchronization of neurons. In this case phase difference is non-zero due to the delaying effect of the synapses. Nevertheless, a one-way path is not robust to perturbations, since information of what happened cannot reach all modules, only those located after the affected module on the path.

We now take the full CPG again and increase the conductance of the inhibitory synapse from the "side" neuron to the promotor neuron from $g = 5$ to $g = 7$. Figures 3.20 and 3.21 show the effect of a stronger inhibition.

There are two different regimes during the evolution of the system. A first regime of approximately the same phase differences, most probably caused by a rapid excitatory synchronization, and a second regime of much higher phase differences. The shift of regime is propagated from p10 to p1, in the sense that it happens in p10 and then succesively in all neurons down to p1. This could be a slower phase adjustment caused by the inhibitory loop. Periods become larger in this second regime, so it is likely that it is inhibition who is causing the effect.

Higher values of the inhibitory synapse display this two regimes behaviour, but activity is much more irregular.

In conclusion, this CPG achieves a stable oscillatory state with constant phase difference between modules. Although this could be achieved with only the one-way excitatory path, the inhibitory loop acts as a redundant phase detector that theoretically could help the robot recover better from perturbations. Furthermore, it has a benefit over the previous symmetrical CPGs in that the direction of displacement is controlled. As a disadvantage, in order to make the robot travel in the opposite direction we would need a replicated set of connections in the opposite direction, each circuit being activated depending on the direction of travel.

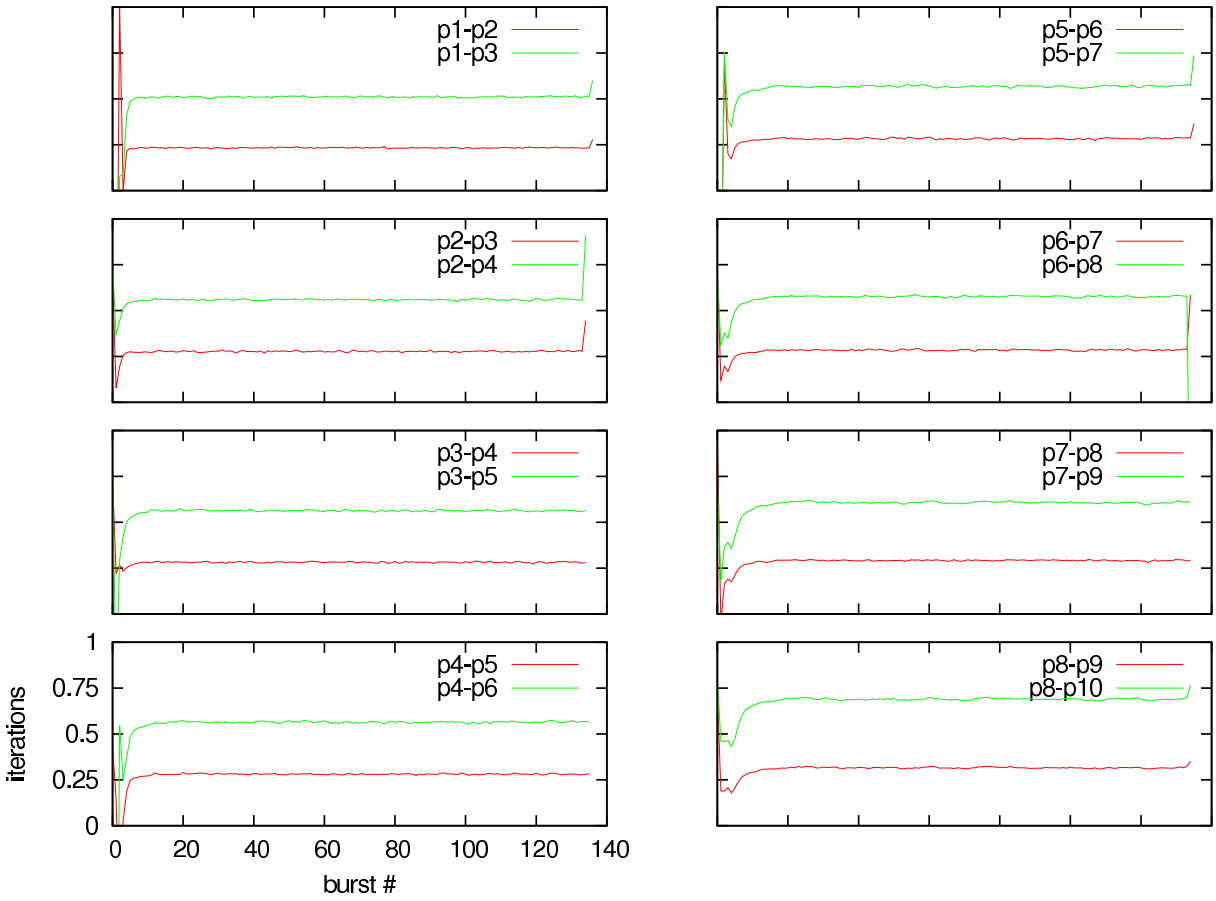


Figure 3.16: *Phase differences for the basic inhibitory loop CPG. Phase differences are very regular accross modules. Border neurons p1 and p10 show a slightly different phase, but not important for the generation of an efficient rhythm. There is a short transient of adjustment at the beginning. This small interval has a small local minimum, as can be observed in p7-p8. This is probably caused by the inhibitory circuit being activated.*

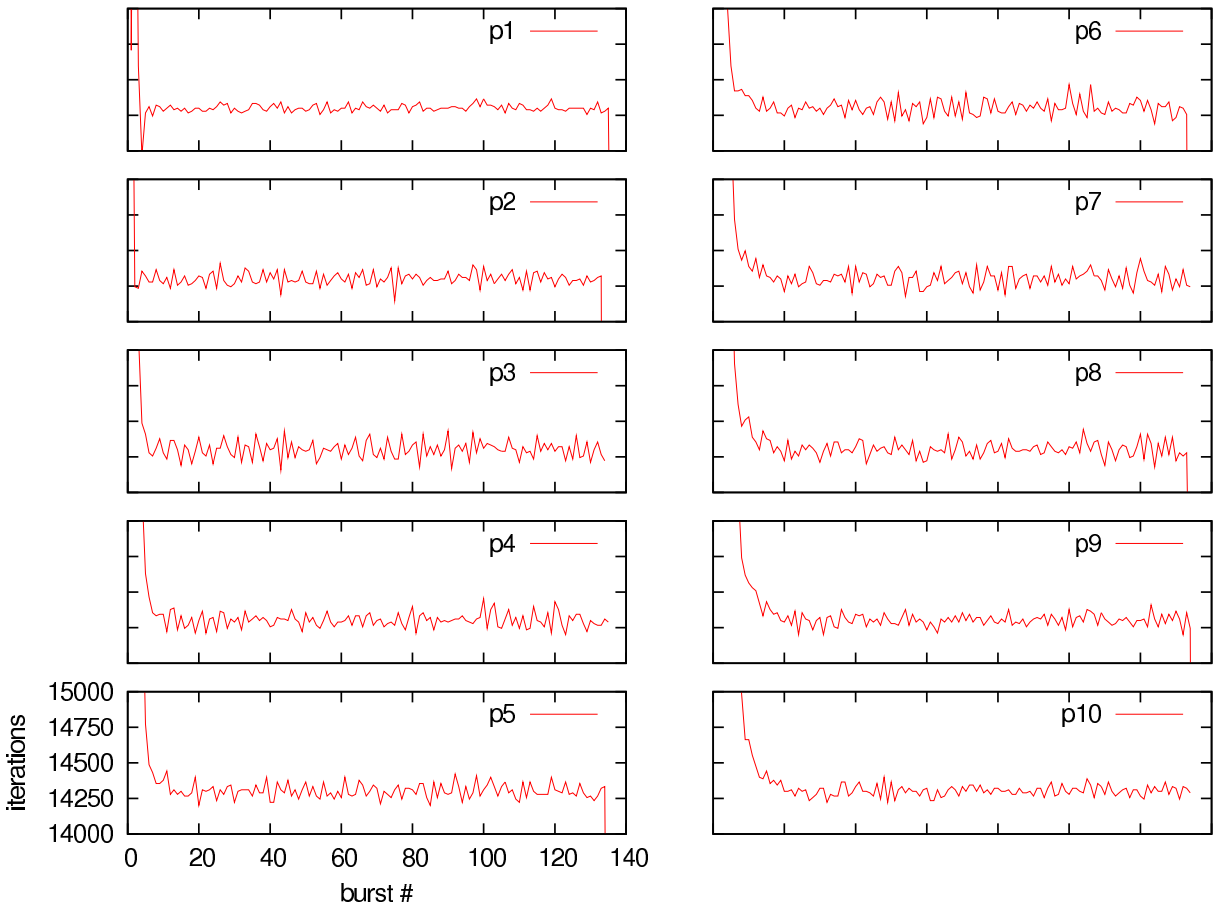


Figure 3.17: *Periods for the basic inhibitory loop CPG. There is a transient interval at the beginning in which frequency is adjusted. All neurons show a lower period (higher frequency) after the transient.*

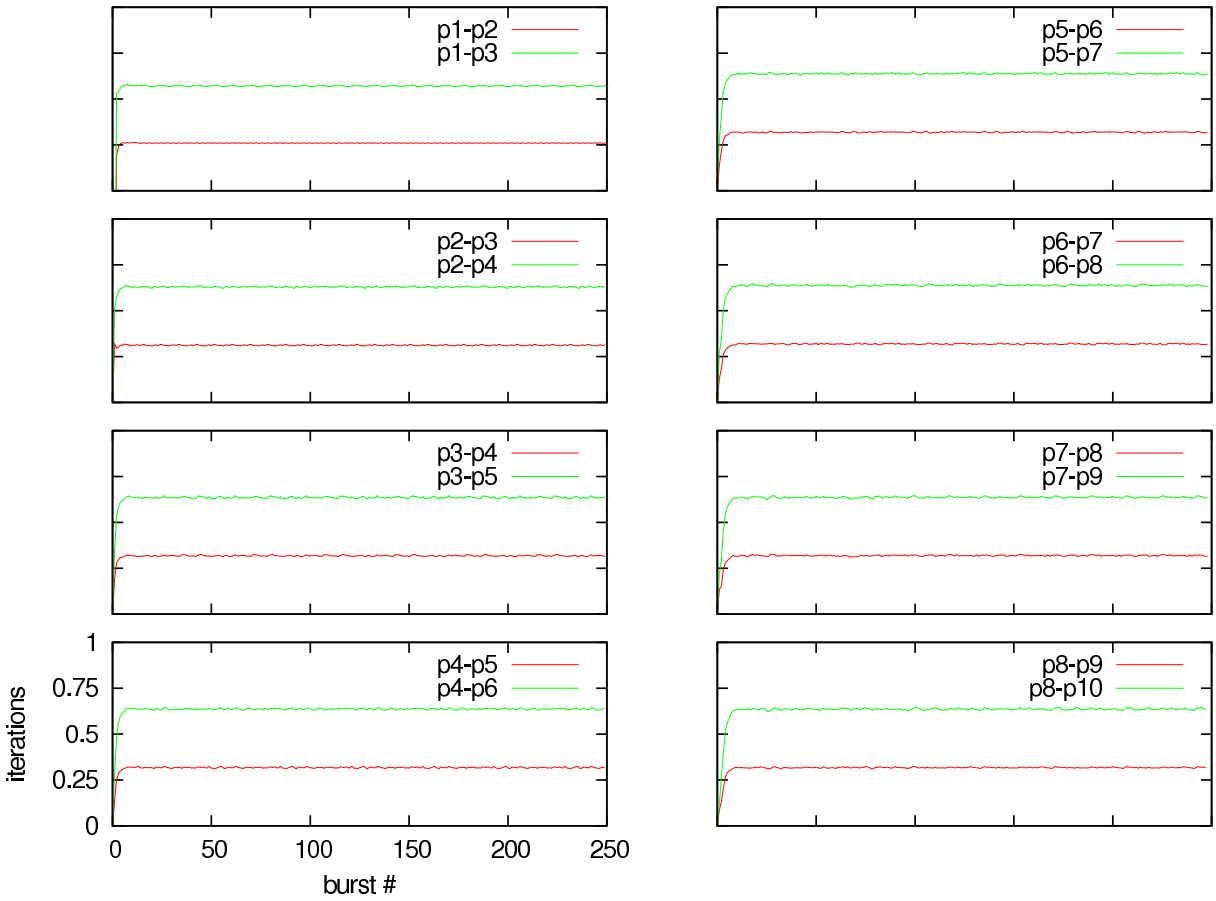


Figure 3.18: *Phase differences for the inhibitory loop CPG, with the inhibitory path suppressed, excitatory path only. Phase differences are larger than in the closed loop case. The initial transient interval does not show the local minimum of the closed loop CPG.*

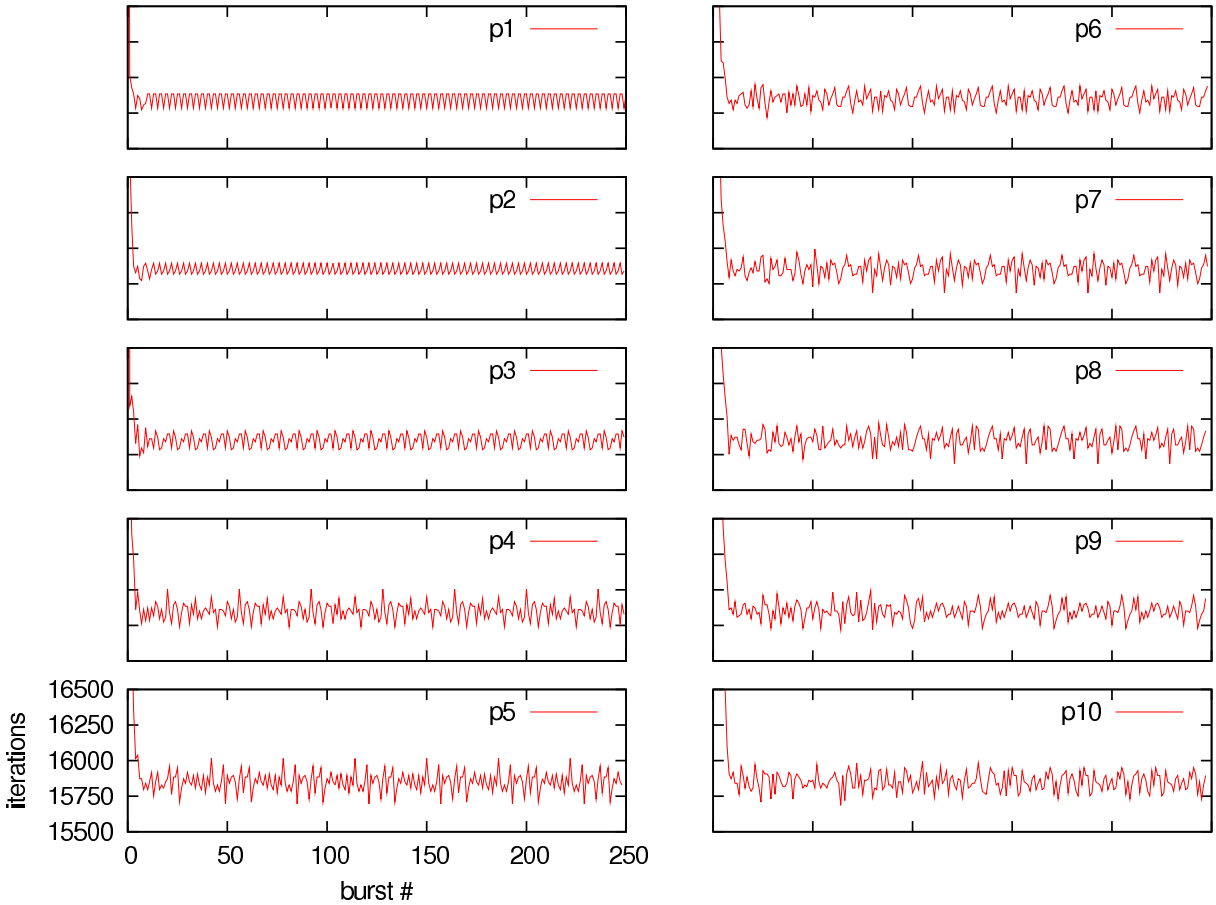


Figure 3.19: *Periods for the inhibitory loop CPG, excitatory path only. Contrary to intuition, periods are now larger than in the closed loop case. In the closed loop case, the inhibitory path shortens the bursts. In this case, bursts are naturally longer, and this explains the longer periods.*

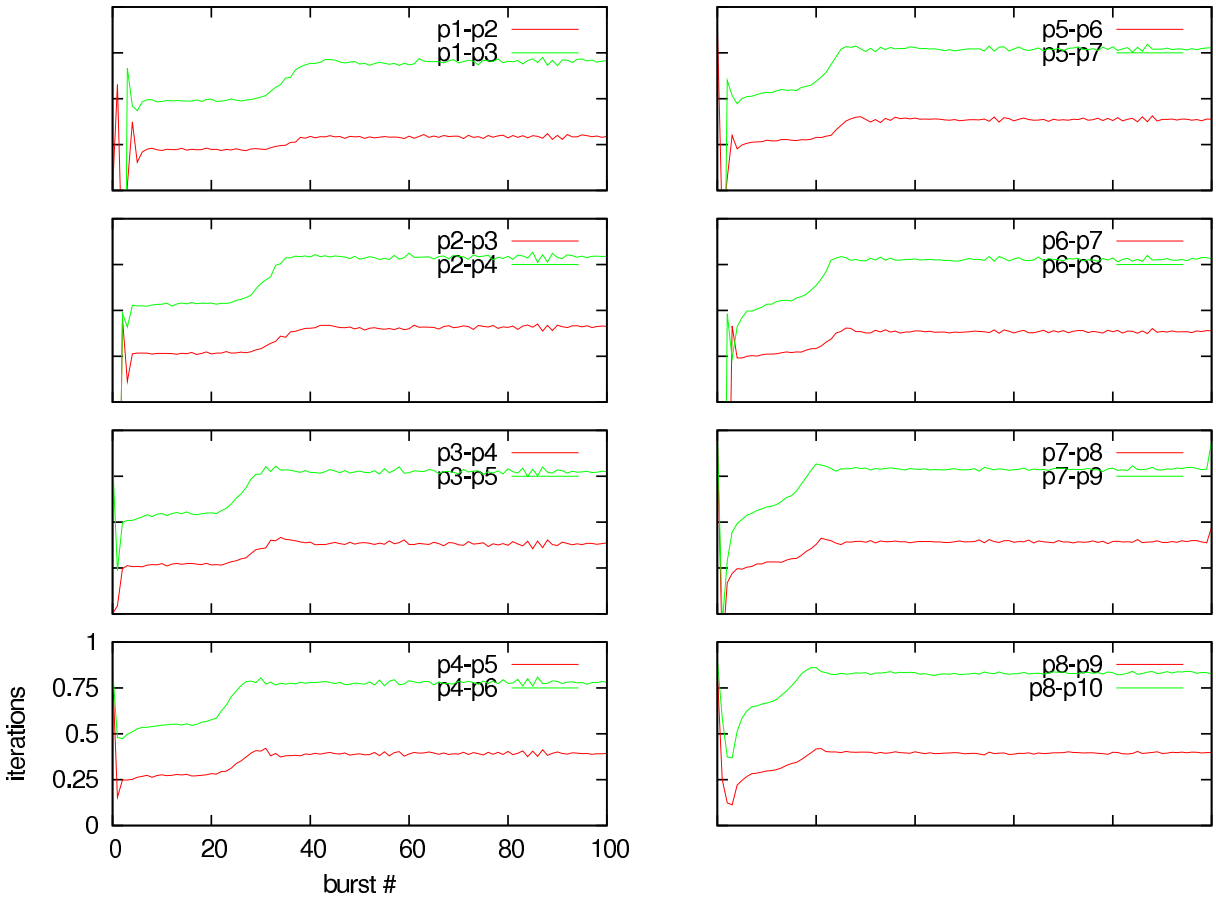


Figure 3.20: *Phase differences for the inhibitory loop CPG, stronger inhibition from "side" to promotor neurons: $g = 7$. The beginning transient interval is longer than in the two previous cases. During the first transient the last modules slowly converge to their final stable state (p8-p9, p8-p10), but earlier modules show a stabilized behaviour (p1-p2, p1-p3). As the last modules increase their phase difference, the loops propagate the effect backwards. This fact is reflected in the onset time of the second regime, with p1-p2 showing this effect almost 20 bursts later than p8-p9.*

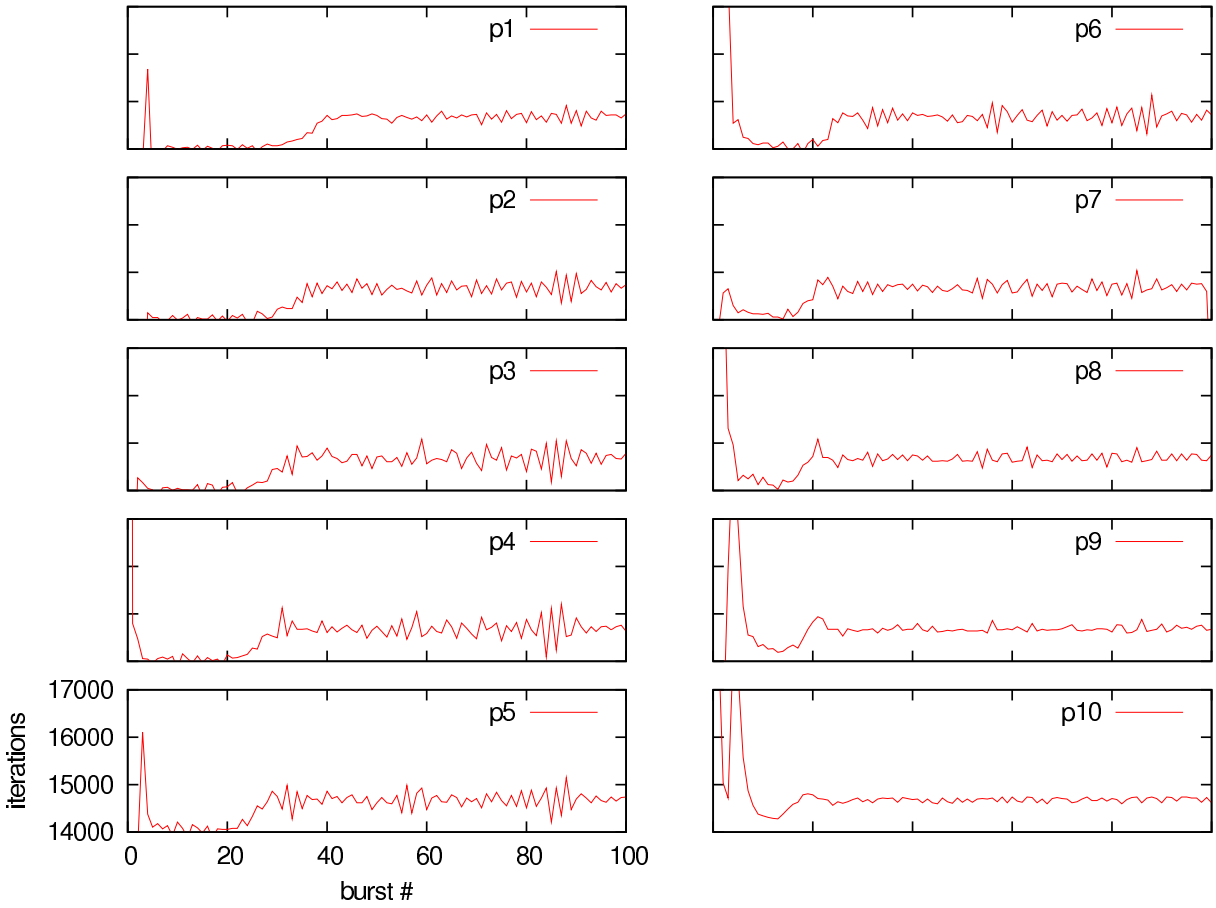


Figure 3.21: *Periods for the inhibitory loop CPG, stronger inhibition from "side" to promotor neurons: periods, like phase differences, show two working regimes too.*

3.5 Push-pull model

3.5.1 Motivation

Many circuits accross the central nervous system show simultaneous excitation and inhibition. Contrary to what it may seem, this is not equivalent to a weaker excitation or inhibition [28].

Figures 3.23 and 3.24 show the behaviour of this scheme.

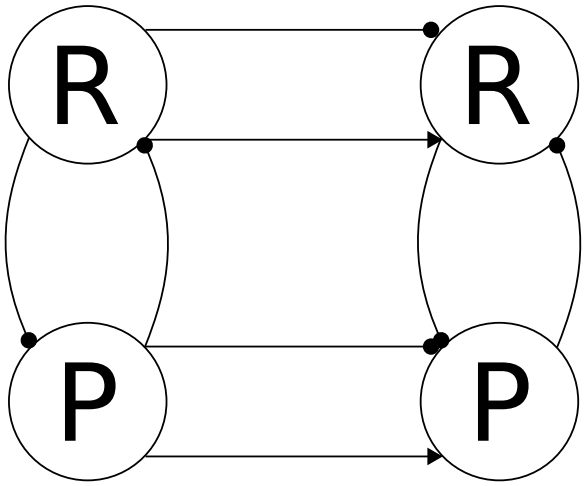


Figure 3.22: *Push-pull model.*

3.5.2 Parameters

- Neurons:
  - $\alpha = 15$, $\mu = 0.001$, $\sigma = -0.33$, $\beta = 0$ and $\sigma_e = 1$.
- Excitatory synapses:
  - $\alpha = 0.5$,$\beta = 10$, $\tau = 0$, $E_{syn} = -7$, $g_{syn} = .75$, $T = 1$, $release_time = 0.01$.
- Ipsilateral inhibitory synapses:
  - $\alpha = 0.5$,$\beta = 10$, $\tau = 0$, $E_{syn} = 8.5$, $g_{syn} = 1.75$, $T = 1$, $release_time = 0.01$.

3.5.3 Evaluation

Synaptic input from one module to its neighbour depends on phase difference. Both excitatory and inhibitory synapses have the same time constants, the only difference being their reversal potential. Recalling equation 2.5:

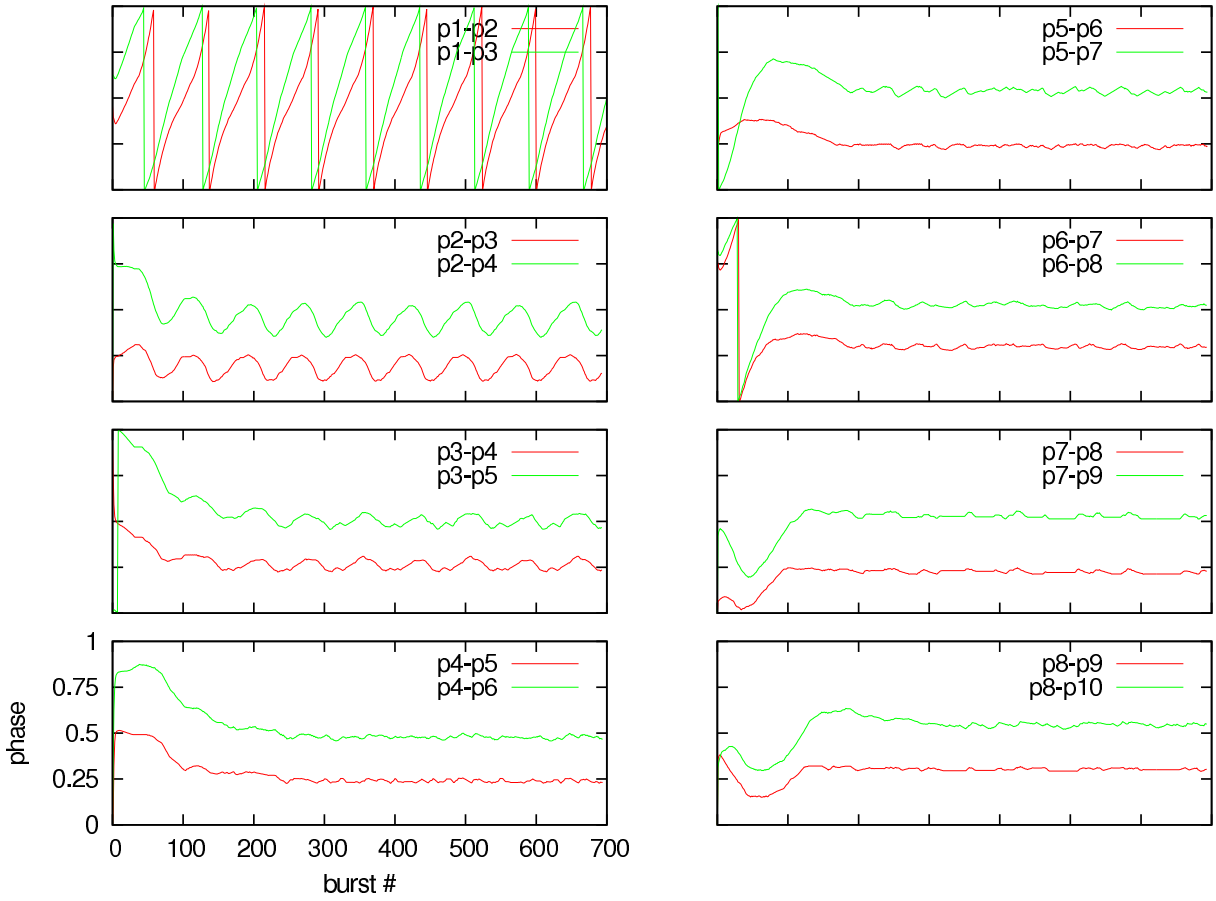


Figure 3.23: *Phase differences for the push-pull model. Modules 2 and 3 oscillate at lower frequencies than module 1 because of their inhibitory input. There is no mechanims that can adjust module 1's frequency, causing the phase drift shown in p1-p2 and p1-p3. Later modules receive input from already slowed down modules, and the CPG is able to maintain stable phase difference (p8-p9 and p8-p10).*

$$I(t) = g \cdot r(t) \cdot (V_{post}(t) - E_{syn})$$

The combined result of both will depend on whether the postsynaptic neuron's potential is closer to the excitatory synapse's E_{syn} or to the inhibitory's. That is, if the postsynaptic neuron is hyperpolarized (at a very negative value) when synapses are active, the end result will be inhibitory (membrane potential will be further from the inhibitory synapse's reversal potential, $E_{syn} = 8.5$ and will contribute more than the excitatory's $E_{syn} = -7$). On the other hand, if it is depolarized (bursting or spiking), its membrane potential will be closer to the inhibitory's reversal potential, thus excitation will contribute more.

So for modules oscillating at the same frequency there is an unstable equilibrium point when they oscillate completely in phase. Then, there will only be excitation. However,

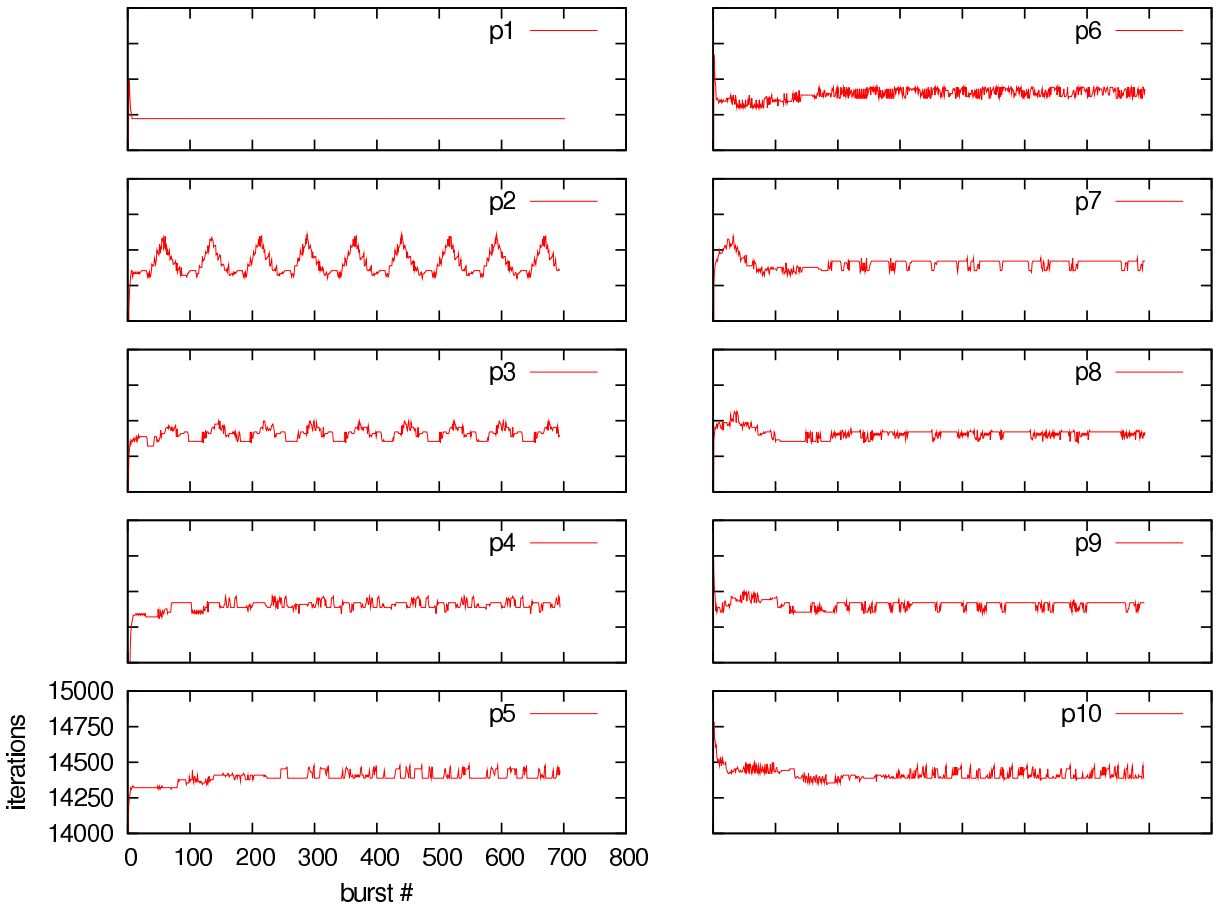


Figure 3.24: *Periods for the push-pull model. P1 has a shorter period, meaning higher frequency due to the abscence of inhibitory input. The net effect of the input to module 2 depends on its phase difference with module 1. Thus, phase drift causes a periodic sequence of excitation and inhibition that affects module 2's frequency. Module 3 suffers the same periodic corrections, but less accute, since module 2 is already slower than module 1.*

inhibition will appear whenever in phase synchronization is not perfect. A stable equilibrium state exists that maintains a constant phase difference, as shown in Figure 3.23.

Let's analyze Figure 3.23. The oscillatory behaviour of the second module can be explained as follows. As already explained, the nature of the input from the first module to the second will depend on their phase difference: it will sometimes be excitatory and sometimes inhibitory. Inhibition decreases the second module's frequency (higher mean period in Figure 3.24) thus causing a phase drift between it and the first one. As they approach in-phase bursting, excitation restores the second module's frequency, but the first module still oscillates faster. Once the first module has overtaken the second one, inhibition reappears and slows down the second module again.

As Figure 3.23 shows, this effect is less accute in later modules. It is because later modules receive input from an already slowed down module, and are able to slow down its successor too. This way, the CPG can reach a stable phase difference.

The key problem here is that there is no mechanism to adjust the first module's frequency to the slower oscillation of its followers. Thus, the need for a non-open [12] topology becomes apparent. The connectivity, however, has proven to be an effective phase maintenance mechanism, given the two adjacent modules oscillate at a similar frequency.

3.6 Bistable intermediate neurons

3.6.1 Motivation

The previously mentioned phenomenon of sequence inversion motivated this organization.

Let's assume that constant phase difference between adjacent modules is not as important as their keeping the right sequence. In order to achieve this, let's see what makes neurons keep a sequence.

In Figure 3.25, two neural activities are shown: p3 and p5, synchronized in anti-phase. Rectangles indicate activity of a neuron, and the intervals labelled "1" and "2" indicate silent episodes between activities. These labels correspond to the two possible types of intervals in which a neuron could fire in order to make a stable sequence. The point is that a third neuron playing a role in the sequence can only fire in *one* of the two intervals, namely only after p3 and before p5 or vice-versa. This is shown in Figure 3.26.

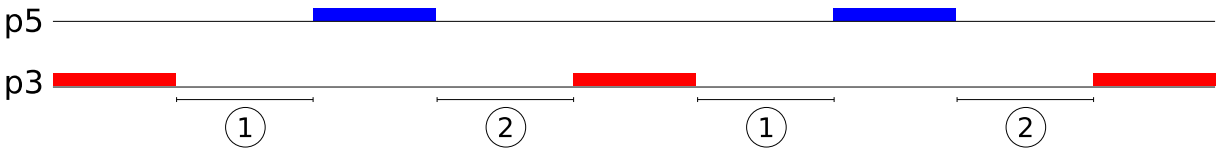


Figure 3.25: *A sequence of two neurons, p3 and p5 oscillating in anti-phase and free intervals (1) and (2) where p4 can fire.*

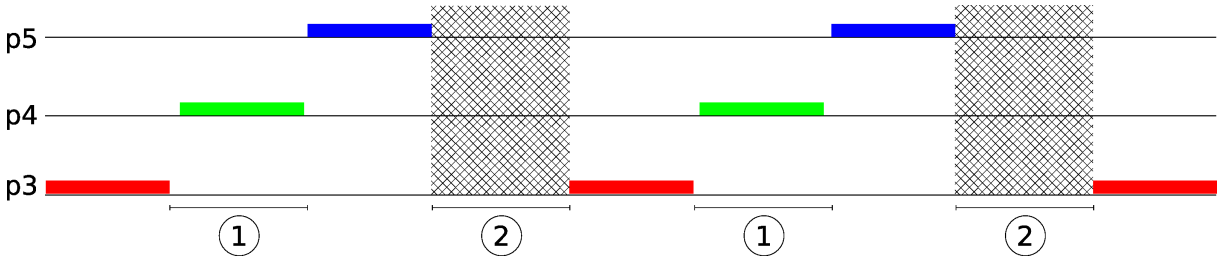


Figure 3.26: *In order to achieve a stable sequence, p4 may only fire in one of the intervals (1) or (2). If it fires in (1), then it can only fire after p3 has fired and before p5 has done it; forbidden intervals for p4 marked grey*

To design a mechanism that prevents a neuron from firing in forbidden intervals, we need a system that is context aware, in the sense that it has to be able to discern what type of interval it is in. This system will strongly inhibit the middle neuron (p4) in forbidden intervals and do nothing in allowed intervals.

Figure 3.27 shows a scheme in which intermediate neurons "B" have been added. These neurons can be in two states: tonic firing or silent. In which state the neuron is depends only on the origin of the last input it has received. That is, if the last input was excitatory, the neuron will be in the tonic spiking regime, *even if the excitatory input has ceased*. Likewise, if the last input was inhibitory, the neuron will be silent, regardless of whether inhibition is still in effect or not.

To be more clear, let's take one "B" neuron and call those neurons from which it receives input *right* and *left*, and let's call *middle* the neuron it inhibits. When *right* bursts, "B" enters the silent mode, meaning the *middle* neuron is allowed to burst between *right* and *left*. When *left* bursts, "B" quickly switches to tonic firing mode, totally inhibiting *middle*'s activity. This way, *middle* has only one chance to burst per cycle.

Bistable neurons are based on the Rulkov map with some modifications. In order to make the neuron bistable, parameter σ is manipulated. Two models have been implemented: in the first case σ is instantaneously changed according to the sign of the input; in the second case, σ is treated as a variable, σ_n, with a governing equation with two stable equilibrium points. This equation makes σ_n switch from one state to the other depending on input, but with a smoother non-instantaneous transition.

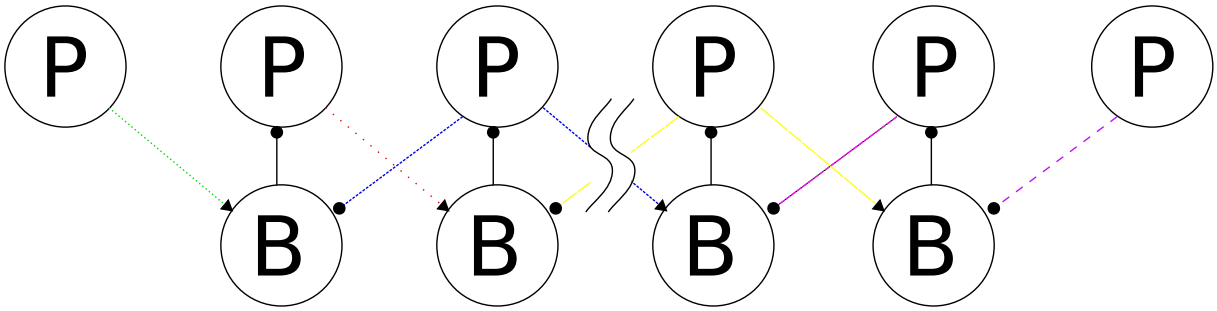


Figure 3.27: *Bistable neurons model (remotors side symmetric, ommited for clarity). Bistable neurons 'B' are either in tonic spiking regime or silent. An excitatory input will put them in tonic spiking. An inhibitory input will drive them to their silent state. When either input ceases, the neuron will remain in the same state until a new input arrives. When in tonic firing, they will completely inhibit their corresponding 'P' neuron. Note the lack of direct interaction between the promotor neurons.*

3.6.2 The bistable neuron model with instantaneous state transition

The bistable neuron model is based on Rulkov's. The neuron is made to oscillate between two states of silent and tonic firing by changing parameter σ depending on synaptic input. The complete equations are 3.1 through 3.4.

$$f(x,y) = \begin{cases} \frac{\alpha}{1-x} + y & \text{if } x \leq 0 \\ \alpha + y & \text{if } 0 \leq x < \alpha + y \\ -1 & \text{otherwise} \end{cases} \tag{3.1}$$

$$x_{n+1} = f(x_n, y_n) \tag{3.2}$$

$$y_{n+1} = y_n - \mu(x_n + 1) + \mu\sigma_n \tag{3.3}$$

$$\sigma_n = \begin{cases} 0.33 & \text{if } I_n > 0 \\ -0.33 & \text{otherwise} \end{cases} \tag{3.4}$$

Due to the opposing nature of the input to bistable neurons (one inhibitory and one excitatory input), the neuron will change its state depending on which presynaptic neuron has fired last. The change in this case will take effect inmediately, since σ is changed as soon as I_n changes its sign.

Figure 3.28 shows the evolution of relative phases of the bistable neurons model in Figure 3.27 with the inmediate bistable neuron model. Modules 2 to 6 have a relatively stable phase difference, considering how unstable periods are (Figure 3.29). There is a propagating wave along them, but is not followed by the other modules. Modules 5 to 10 show *parity syncronization*. It can be observed how phase differences with second neighbours p6-p8, p7-p9 and p8-p10 are around 0 (or equivalently 1), and phase differences with first neighbours p6-p7, p7-p8, p8-p9 oscillates at around 1/2, which corresponds to parity synchronization mode 2 described earlier.

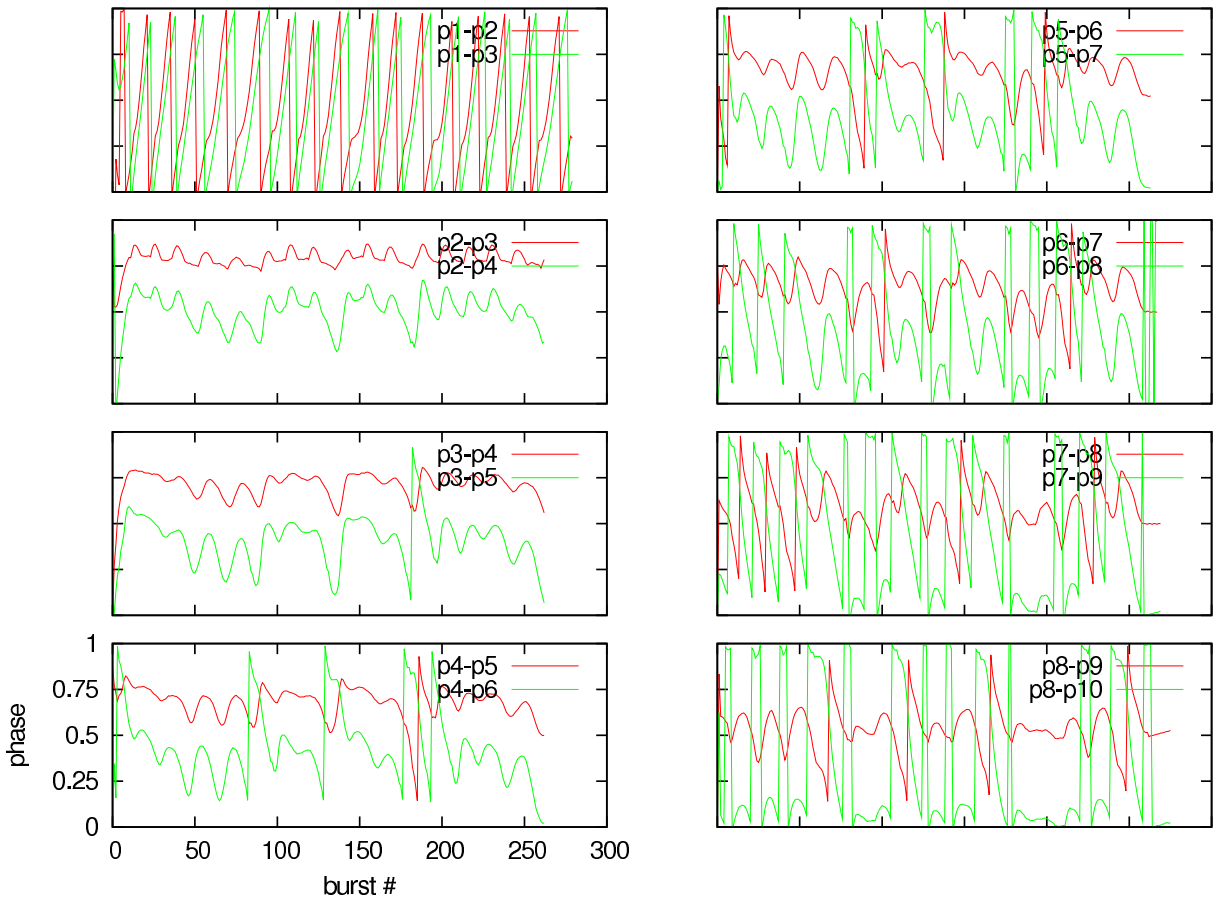


Figure 3.28: *Phase differences for the bistable neurons model with instantaneous state transition. P1 receives no input, oscillating at its nominal frequency, higher than P2 and P3. Neurons P6 to P10 show a fuzzy kind of synchronization that we term "parity synchronization mode 2", with first neighbours synchronized approximately in anti-phase.*

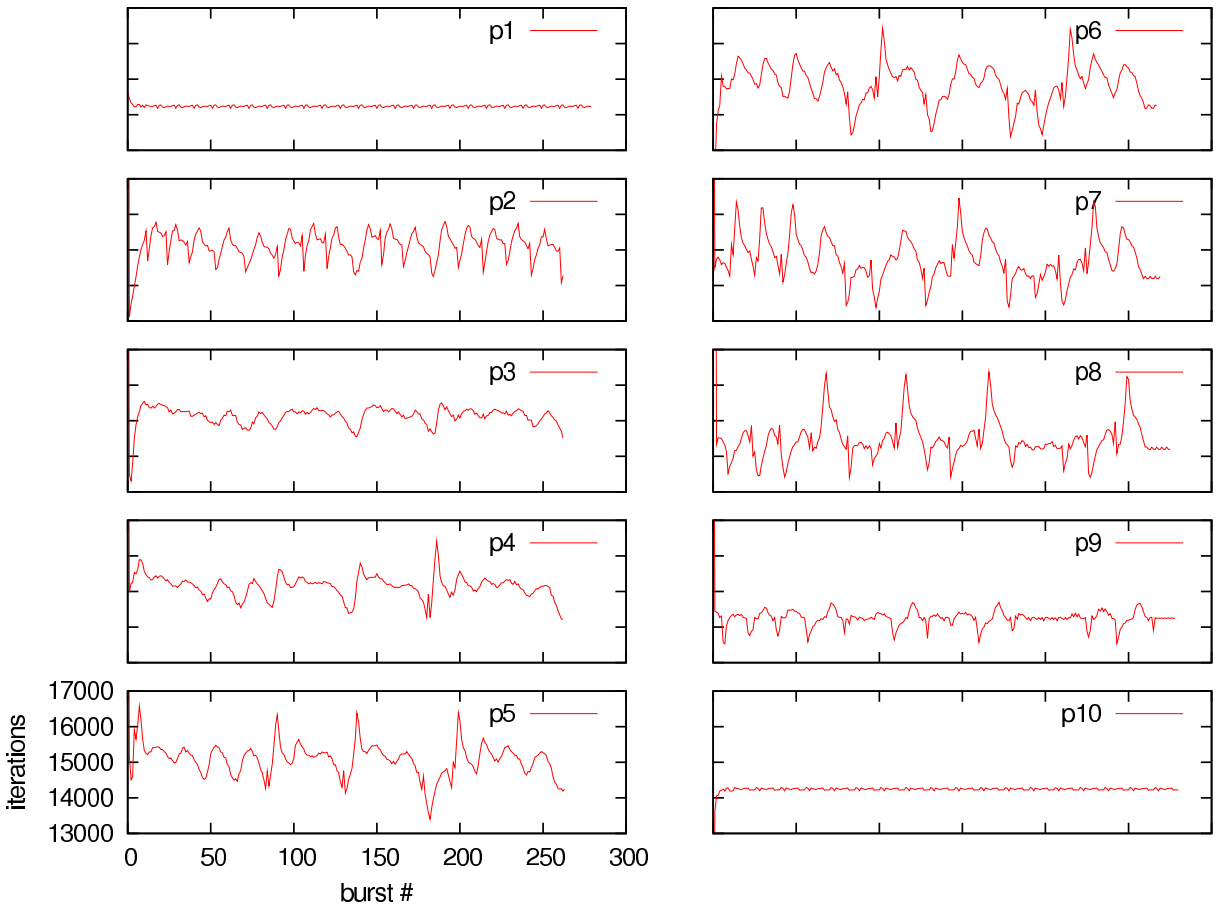


Figure 3.29: *Phase differences for the bistable neurons model with instantaneous state transition. Neither of the border neurons receives input, thus oscillating freely at their nominal frequency. Activity is noticeably irregular.*

3.6.3 The bistable neuron model with delayed state transition

Instead of having σ sharply change from one value to the other depending on the sign of the input current, we now substitute this threshold in equation 3.4 with a map equation in 3.8. The complete equations are 3.5 through 3.8.

Now σ_n has a governing equation (3.8) with two stable equilibrium points. The role of I_n in this equation is to raise or lower the curve so that one of the stable states disappears and σ_n is forced onto the other stable state.

This dynamical approach displays a transitory period that introduces a delay in the response between the presynaptic neuron's activation and the bistable neuron's changing its state.

$$f(x,y) = \begin{cases} \frac{\alpha}{1-x} + y & \text{if } x \leq 0 \\ \alpha + y & \text{if } 0 \leq x < \alpha + y \\ -1 & \text{otherwise} \end{cases} \tag{3.5}$$

$$x_{n+1} = f(x_n, y_n) \tag{3.6}$$
$$y_{n+1} = y_n - \mu(x_n + 1) + \mu\sigma_n \tag{3.7}$$
$$\sigma_{n+1} = p \cdot (\tanh(2\sigma_n) + I_n) \tag{3.8}$$

As observed in Figure 3.30, phase differences are now much more stable, as well as periods (Figure 3.31). There is, however, an oscillatory behaviour and some irregularities at the border modules. This is because of the lack of feedback from the rest of the body. Since all modules are somehow inhibited except for the border modules, their frequency is higher than the rest, causing a drift in their relative phases that causes periodic modifications of the overall rhythm.

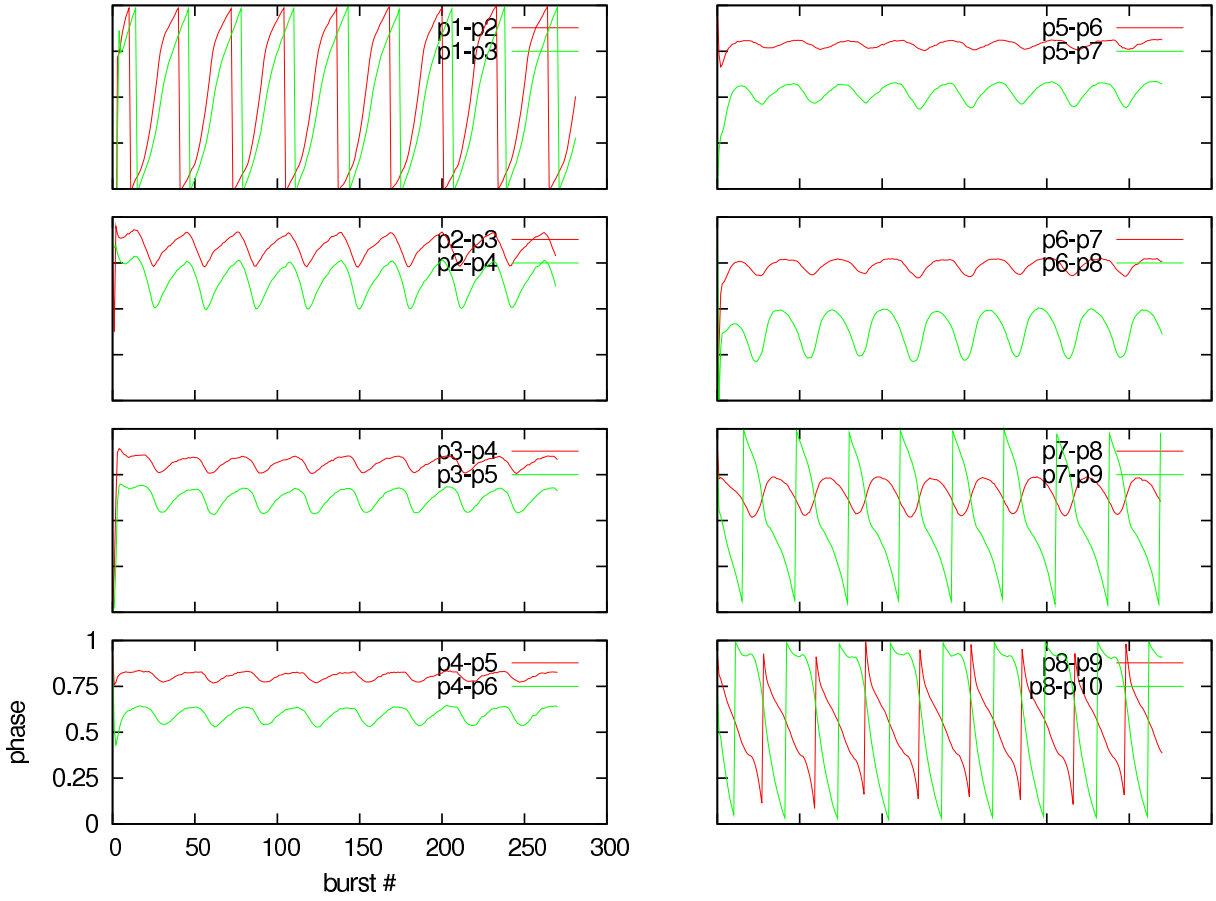


Figure 3.30: *Phase differences for the bistable neurons model with delayed state transition. Activity is much smoother than with instantaneous transitions. Border effects still noticeable.*

3.6.4 Counteracting border effects

To counteract border effects we add a single synapse from two interior neurons to border neurons. This should have the effect of coordinating border activation rhythms by making

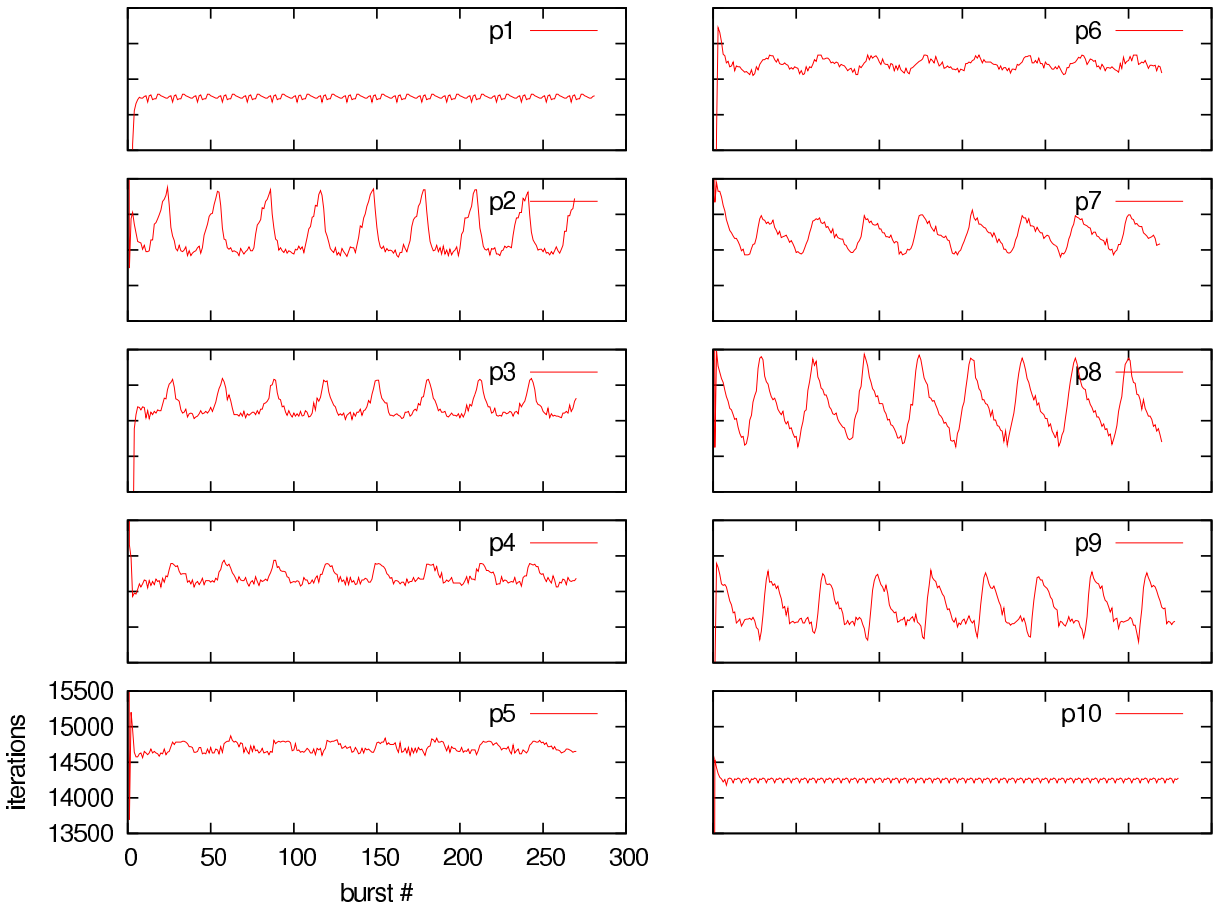


Figure 3.31: *Periods for the bistable neurons model with delayed state transition. Activity is much smoother than with instantaneous transitions. Border effects still noticeable.*

the topology *non-open* [12] and thereby adjusting the borders' frequencies. The diagram is shown in Figure 3.32. The synapses depart exactly from the borders' second neighbours, so that these two neurons define a context for the border modules' first neighbours.

Figures 3.33 and 3.34 reflect this stabilizing effect of making the topology *non-open*. Phase differences are however still not constant along the body, with module 10 displaying a slightly deviated phase difference.

3.6.5 Evaluation

In conclusion, this model has proven to generate a stable firing sequence provided that border neurons receive feedback in order to adjust their frequencies.

The mechanism that coordinates the sequence does not operate through small frequency adjustments, like the previous CPGs, but through total inhibition of neurons. By letting promotors fire only on allowed intervals the direction of travel is established and maintained across modules. Side neurons with delayed state transitions are responsible

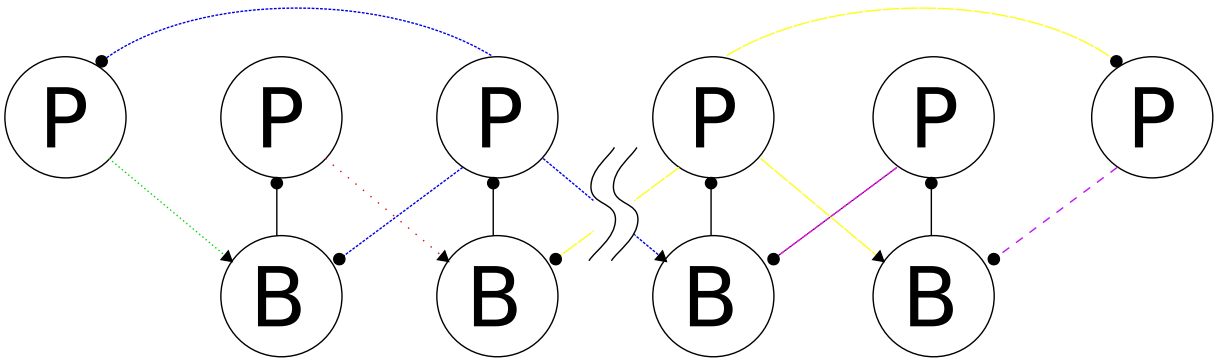


Figure 3.32: *Bistable neurons with a non-open topology to cancel border effects caused by border neurons not having any input. (remotors side symmetric, ommited for clarity)*

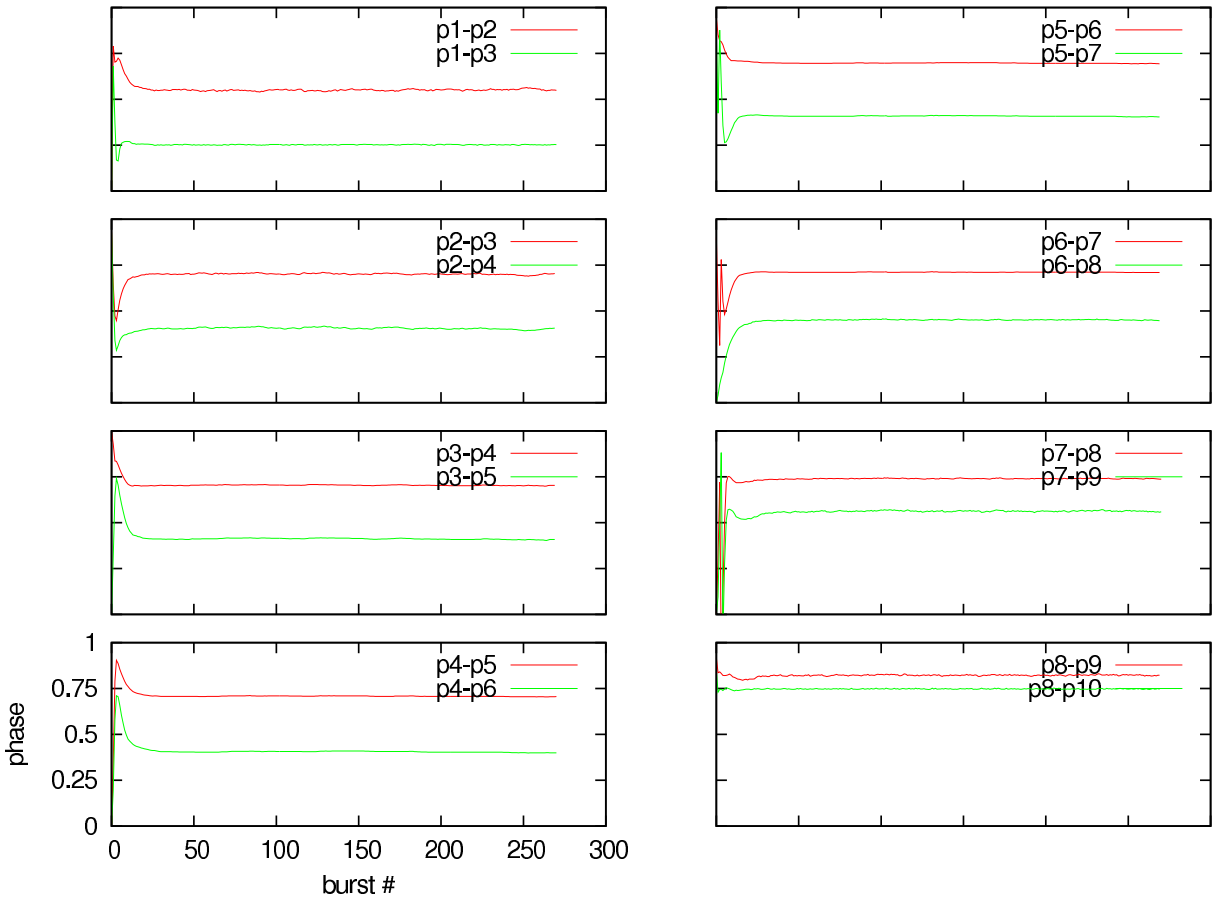


Figure 3.33: *Phase differences for the bistable neurons model with delayed state transition and border effects compensation. P1 and P10 are now synchronized with the rest of the neurons. The synapse to P10 causes a slightly deviated behaviour.*

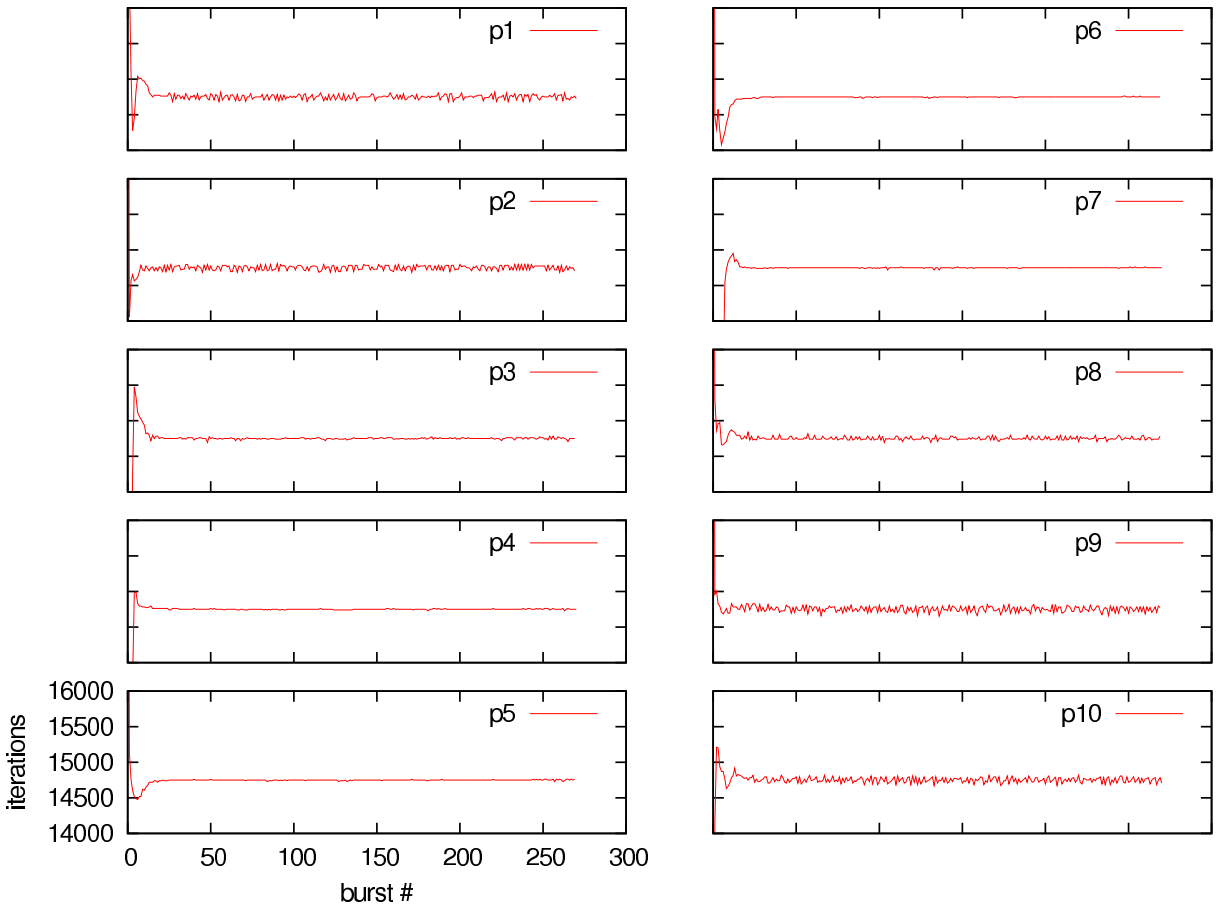


Figure 3.34: *Periods for the bistable neurons model with delayed state transition and border effects compensation. The two new synapses reduce border neurons's frequency and the overall rhythm is stabilized.*

for maintaining an efficient phase difference.

4

Discussion

Several approaches to the task of generating and coordinating a rhythmic movement for a modular robot have been proposed. Here we discuss what has been learnt from them.

4.1 CPG summary

Symmetric inhibition. The first proposed CPG was the "symmetric inhibition". The idea is very simple: connect the neurons with inhibitory synapses to prevent in-phase bursting and see if they can burst in a stable sequence.

We have not studied whether this CPG always finds a stable firing sequence regardless of initial conditions, but it can indeed find a stable sequence for some of them.

This circuit adjusts itself by performing small corrections in neurons' frequency. This way phase difference can be adjusted by drift, that is, one neuron oscillating slower than its neighbours and letting them "drift away". As a result of this slow divergence, there is a long transient interval until a stable state is reached.

Drift may cause "sequence inversion", when one neuron overtakes its neighbour, meaning that the direction of the firing sequence may be reversed. Since the proposed circuit lacks a mechanism to impose a particular direction of travel, the final direction once a stable state has been found is unpredictable.

This, however, means that, if a suitable mechanism to define the direction of travel is developed, like setting more specific initial conditions rather than random ones, the same set of synapses could be used for both forward and backward travel. Another mechanism would be needed for "on the fly" change of direction, nevertheless.

The "symmetric inhibition" CPG is a good candidate if the goal is to maintain an already established firing sequence, in both directions. However, other mechanisms should be used to start the sequence.

Inhibitory loop. This CPG works by correcting one module's frequency depending on its phase difference with its successor. Through a direct synapse from one module to

its neighbour, adjustments can be made to the successor module and through the loop, adjustments are also made to the predecessor module.

The CPG is designed to work in a particular direction. The circuit is capable itself both of maintaining the direction of travel as well as of establishing it in the first place. The transient interval is relatively short, depending on synapse conductance. Real experience is needed to determine if the transient is tolerable or indeed too long.

The "inhibitory loop" CPG is a very stable CPG, capable of starting and maintaining a stable sequence. The loop enables bidirectional adjustment to successor and predecessor modules.

Push-pull. The "push-pull" CPG is a very simple circuit, that nevertheless achieves a very stable phase difference between modules oscillating at similar frequencies.

This CPG works as a phase detector. The net effect of the two synapses, one excitatory and one inhibitory, projected onto the successor module depend on the exact phase of the cycle on which the postsynaptic neuron finds itself. This way, modules can be speeded up or slowed down until the desired phase difference is achieved.

The projected synapses go only one way. This means that information flow is unidirectional. Predecessor modules cannot process information coming from later modules.

One side effect of unidirectional information flow is that the first module oscillates faster than the rest. Since it does not receive any input, its frequency is unaffected, being it its nominal frequency. All other modules oscillate always at a lower frequency, so a stable phase difference (between the first modules) is impossible to achieve.

The "push-pull" CPG is able to achieve stable phase differences provided that adjacent modules oscillate at approximately the same frequency. As it is presented here, the lack of feedback to earlier modules prevents the adjustment of the first module's frequency to its successor's and causes periodic variations in frequency of the second and third modules.

Bistable neurons. The "bistable neuron" CPGs work not through frequency adjustment, but through complete inhibition of neurons. The endogenous rythmic nature of neurons provides for an intermitent inhibitory activity. Topology then drives the system in order to create a wave of succesive inhibitions and activations, defining allowed intervals in which promotor and remotor neurons may fire. This mechanism defines a consistent direction of travel and prevents sequence inversion.

The effect of strong inhibition is to "erase" the past, setting the system in a given state independent of initial conditions. As a result, this CPG might be very resilient to perturbations.

Lack of feedback to border neurons again prevents complete synchronization. However, we provide a model with two synapses that counteract this effect. As shown, just by establishing these two synapses that correct the border modules' frequencies, the CPG is able to achieve a constant phase difference among all of its modules.

4.1.1 Features

Not all CPGs display the same set of behaviours. This table tries to summarize what they can and cannot do:

	Reversible	**Bidirectional information flow**	**Free of border effects**	**Fast convergence**
Symmetrical inhibition	Y	Y	Y	N
Inhibitory loop	N	Y	Y	Y
Push-pull	N	N	N	N
Bistable neurons (without border synapses)	N	Y/N	N	Y
Bistable neurons (with border synapses)	N	Y/N	Y	Y

Reversibility is here considered a feature, but it must be noted that the only reversible CPG is also the only one that is not capable of defining a direction of travel.

The "bistable neurons" CPGs have bidirectional information flow indirectly. Promotor [remotor] neurons are not directly connected to each other, but one module can affect both of its neighbours through their bistable 'B' neurons.

4.1.2 Complexity

Another simple way to compare the CPGs is to sum the number of neurons and synapses and divide it by the number of modules. We call this naive measure "complexity":

$$C = \frac{Neurons + Synapses}{Modules} \tag{4.1}$$

Considering all experiments have been carried out on a robot with 10 modules, these are the results (only the promotor side is measured, since all CPGs are symmetric):

CPG	**Complexity**
Symmetrical inhibition	2.8
Inhibitory loop	5.5
Push-pull	2.8
Bistable neurons (without border synapses)	4.2
Bistable neurons (with border synapses)	4.4

As expected, CPGs with higher complexity yield the most stable results, and CPGs with similar complexity have similar capabilities. For example, "push-pull" and "symmetrical inhibition" have similar complexity values. It may seem that their results are very different, but one should consider the tradeoffs of each CPG: while "push-pull" achieves very stable phase differences, it lacks the ability to be reversible, as is the case for the latter.

Likewise "inhibitory loop" provides stable phase differences, bidirectional information flow, does not suffer from border effects and has the highest complexity. The bistable neurons have almost the same set of features except for the bidirectionality of information flow, since promotor [remotor] neurons are not directly connected among them.

This yields a very interesting question: can features be quantified? Is there a minimum complexity that a CPG must have to display a certain set of features? Intuition says it must be so. Moreover, it does not take the same amount of resources to counteract border effects (only two synapses in "bistable neurons") as to implement bidirectional information flow, which would be proportional to the number of modules.

4.2 Symmetry

In general, it could be said that symmetry is to be avoided. This is not to say that no symmetry should appear, rather that it must be broken at some level. If topology is symmetric, like the first symmetrical model proposed, then initial conditions must break the symmetry. This CPG has the basic ability of adjusting itself to a stable sequence but has no mechanism to determine the direction of movement, so both directions are equally likely given random initial conditions.

If activation of the modules is symmetric, that is, in-phase or anti-phase synchronized, there will not appear any propagating wave that makes the robot undulate, but rather there will be a standing wave. In conclusion, activations must be periodic but not symmetric, i.e., there must be a constant phase difference accross modules. Otherwise the generated movement may not be efficient. *Parity synchronization* modes shown above are an example of symmetry to avoid.

4.3 Coupling strength

Weak coupling is a mechanism to exchange information among modules without *severely* altering the receiver's behaviour. It allows each module to keep its activity seemingly unaffected and independent while at the same time coordination is being "negociated". The disadvantage is that for large systems convergence can be very slow if initial states are sufficiently different.

On the other hand, strong inhibition can sharply change the receiver's activity. This mechanism allows for a faster convergence to a stable state, independent of initial conditions, and a faster response to changes. This is also a good mechanism to prevent against perturbations, since strong coupling has a resetting effect that *erases* the past, thus isolating perturbations in a small time window.

Probably the best approach is to combine both mechanisms and have different connectivities acting at different times: one with stronger coupling to drive the system to a known state (for starting and stopping the movement, or to recover from perturbations) and a weaker system to maintain activity once the stronger one has been shut off.

4.4 Coupling sign

While our first approach was to use inhibitory synapses only, this work has shown that excitatory coupling can also be useful.

Inhibitory synapses act to prevent in-phase synchronization and regulate oscillation frequency. In general, when one module receives an inhibitory current, its frequency is decreased. This is the cause for phase drift in the absence of feedback to the presynaptic neuron.

In some cases, however, inhibition may cause an increase in frequency. If the timing is right, inhibition may shorten a neurons' bursts but leave its recovery process unaffected, thereby shortening bursting period.

Excitatory synapses, on the other hand, promote in-phase synchronization. Taking advantage of the delayed effect of chemical synapses, excitatory coupling can be used to achieve a constant delay between coupled modules.

The push-pull model has shown that these types of coupling are not exclusive, nor are they equivalent to a weaker version of one of them when used simultaneously. Indeed, using both simultaneously we have constructed a stable phase detector.

4.5 Topology

It is very important that all modules share information, that is, send *and* receive stimuli. CPGs based on inhibition will suffer border effects if border neurons do not have any feedback. If this is the case, they will not have any means to adapt their frequency of oscillation to that of the rest of the modules. As a consecuence, the system will not be able to achieve stable phase differences among them.

We have shown that it is possible to coordinate modules on a global scale by using first neighbours only connections. This is however not very resistant to failure since malfunction in one module will comletely break the chain of communication. Projections to further modules and redundant communications will certainly provide richer dynamics and better fault tolerance.

5

Future work

The first obvious step is to implement these models in a real robot. All models have been tested in a simulator, but several details have been left out.

One of them is that motors do not react instantaneously. The servomotors with which the actual robot is built receive a command specifying the desired position to which it should rotate. Then, a PIC controller adjusts rotation speed depending on the distance to the desired angle.

This poses two questions. The first one is if the models implemented on a computer can coordinate such a robot built on such motors. The current approach is to simulate the models on a computer and send the corresponding commands to the motors, without receiving any feedback on their position. This question amounts to whether we can adjust our simulation time so that motor movements can be considered instantaneous.

The second question is, given that the above fails, how we can integrate the information about motor position in the coordination of the movement. This links with one of our work lines which is integration of sensory feedback.

Once we have a model that effectively coordinates the real robot, the next step will be to integrate information about the environment and the robot itself in the generation of motor sequences. The goal is that the robot must be able to generate different behaviours depending on external conditions (terrain, unforeseen perturbations) as well as on internal conditions (malfunction, non-instantaneous movements).

Along this line we will introduce multicoding neurons. These are neurons that can change their interspike interval distribution, using it as a signature. Receivers that are capable of decoding this distribution will be able to selectively react to messages and even discriminate among senders. This will provide the robot with a much richer sensitivity to changing conditions due to a richer context awareness.

Finally, we will explore different topologies for the robot. This work has only considered propagating waves in one plane. Actually, the robot can be configured so that some modules oscillate in one plane and others in a perpendicular plane, allowing for a much richer set of movements. Also, this work has only considered modules lined up in one dimension. It would be interesting to explore bidimensional or tridimensional arrangements, symmetric and asymmetric, but this is a larger term goal.

Appendix A

A C++ library for simulation of dynamical networks

Let's illustrate how the library can be used.

Suppose we want to simulate a Hindmarsh-Rose [29] neuron, using a Runge-Kutta 4th order integrator for one second using a timestep of $0.001s$.

First, we must include the necessary files:

```
#include <models/HindmarshRoseModel.h>
#include <integrators/RungeKutta4.h>
#include <wrappers/SystemWrapper.h>
#include <wrappers/IntegratedSystemWrapper.h>
```

The first two included files are readily understandable. The other two may require some more explaining.

The class HinmarshRoseModel is a brief prescription of the Hindmarsh-Rose model:

```
template <typename Precission>
class HindmarshRoseModel  : public NeuronBase<Precission>{
public:
  typedef Precission precission_t;

  enum variable {x, y, z, n_variables};
  enum parameter {e, mu, S, n_parameters};

public:
  void eval(const Precission * const vars, Precission * const params,
      Precission * const incs) const
  {
    incs[x] = vars[y] + 3.0 * vars[x] * vars[x] - vars[x] * vars[x] *
        vars[x] - vars[z] + params[e] + SYNAPTIC_INPUT;
    incs[y] = 1 - 5.0 * vars[x] * vars[x] - vars[y];
    incs[z] = params[mu] * (-vars[z] + params[S] * (vars[x] + 1.6));
  }
```

```
};
```

It contains no data members: this class cannot store any value neither for the variables nor for the parameters. Model classes must be simple by design, so that adding a new model has as little implementation details as possible.

It is through *wrapping* this class that new functionality can be added. For instance, the class SystemWrapper adds storage functionality and methods to set and get the values of the variables and parameters of the model:

```
typedef HindmarshRoseModel<double> Model;
typedef SystemWrapper<Model> System;
typedef IntegratedSystemWrapper<System, RungeKutta4> Neuron;
```

Here you see why they are called wrappers. In this case we use two wrappers: SystemWrapper to add storage access funcionality and IntegratedSystemWrapper to add stepping funcionality.

Here is the main body of our program:

```
int main()
{
  Neuron::ConstructorArgs args;

  args[Neuron::e] = 3.0;
  args[Neuron::mu] = 0.0021;
  args[Neuron::S] = 4;

  Neuron n(args);

  const double step = 0.001;

  for(int i = 0; i < 1000; i++){
    n.step(step);

    cout << n.get(Neuron::x) << endl;
  }

  return 0;
}
```

A.1 Using synapses

To add synapses (diffusion synapses, in this example) to our simulation we must add these lines to the head of our program:

```
#include <include/DiffusionSynapsis.h>
```

```
typedef DiffusionSynapsis<Neuron, Neuron, RungeKutta4> Syn;
```

The three template parameters are: type of the presynaptic neuron, type of the postsynaptic neuron and integrator type.

We can now instantiate it and step it:

```
...
  Neuron n(args);
  Neuron n2(args);

  Syn::ConstructorArgs synargs={0.5, 10, 0, 9, 1.5, 1, 0.01};

  Syn syn(n, Neuron::x, n2, Neuron::x, synargs);

  const double step = 0.001;

  for(int i = 0; i < 1000; i++){
    syn.step(step);

    n.step(step);
    n2.step(step);

    cout << n.get(Neuron::x) << ' ' << n2.get(Neuron::x) << endl;
  }

  return 0;
}
```

Here we see another way of filling out the constructor arguments. This way may be confusing, because the name of the parameter is not explicitly specified, but it comes in handy for long lists of arguments like this one.

When constructing the synapse object we need to specify the pre- and postsynaptic neurons as well as the variables to be measured. Once the object is constructed, it has a step method like any other dynamical system of the library.

Note that synapses should be stepped **before** any other element in the network, so that postsynaptic neurons know their right input when stepping.

A.2 Simulating iterated maps

Discrete maps are not different from differential equations from this library's perspective. As before, we need to include the model, the wrappers we need and the integrator to use:

```
#include <models/RulkovMapModel.h>
#include <integrators/Stepper.h>
```

```
#include <wrappers/SystemWrapper.h>
#include <wrappers/IntegratedSystemWrapper.h>
```

Note that we are using the Stepper integrator now. The rest of the simulation is exactly the same, except for one point: the step method still accepts a timestep parameter, but it is not taken into account, the model always steps one time unit.

A.3 Adding new models

Adding a new neuron model is really easy using the following template:

```
#ifndef MODEL_H_
#define MODEL_H_

#include <cmath>
#include ''../include/NeuronBase.h''

template <typename Precission>
class Model  : public NeuronBase<Precission>{
public:
  typedef Precission precission_t;

public:
  void eval(const Precission * const vars, Precission * const params,
      Precission * const incs) const
  {
  }
};

#endif /*MODEL_H_*/
```

One only needs to add:

- A variables enum, including the names of all involved variables, terminated with "n_variables".
- A parameters enum, including the names of all involved parameters, terminated with "n_parameters".
- Equations to the eval method. For an equation of the form $\dot{x} = \alpha x + \beta y$ one should write

```
incs[x] = params[alpha] * vars[x] + params[beta] * vars[y];
```

If the model is an iterated map, it is almost the same, except for the eval method:

- For a map of the form $x_n = \alpha x_{n-1} + \beta y_{n-1}$ write:

```
incs[x] = params[alpha] * vars[x] + params[beta] * vars[y];
```

Now we calculate the next point directly, not the increment. The idea is that the statement always ressembles the mathematical expression.

A.4 Available wrappers

These are all the wrappers available at the moment in the library.

A.4.1 SystemWrapper

When applied to a model class "Model", this wrapper adds a member array called "m_variables" of size "Model::n_variables" and another one called "m_parameters" of size Model::n_parameters".

Besides, the following methods are defined:

```
precission_t get(variable var) const
{
  return m_variables[var];
}

void set(variable var, precission\_t value)
{
  m_variables[var] = value;
}

precission_t get(parameter param) const
{
  return m_parameters[param];
}

void set(parameter param, precission_t value)
{
  m_parameters[param] = value;
}
```

A.4.2 IntegratedSystemWrapper

This wrapper takes a system class and an integrator class as template arguments and provides a method to step the model:

```
void step(precission_t h)
{
  /* Allow system specific step actions
   */

  pre_step(h);

  Integrator::step(*this, h, System::m_variables,
      System::m_parameters);

  post_step(h);
}
```

A.4.3 TimeWrapper

This class adapts models working at different time scales. At construction time it receives an argument specifying how many calls to the "step" method will cause an actual step of the model:

```
TimeWrapper(ConstructorArgs const &args, int substeps_per_cycle = 1)
    : m_substeps_per_cycle(substeps_per_cycle), m_count(0),
    IntegratedSystem(args)
{
}
```

Due to this restriction, this wrapper should be applied the last, because other wrappers do not know yet how to receive this argument in their constructor (a future implementation will include the solution described in [27]).

It also provides the following step method:

```
void step(precission h)
{
  m_count++;

  if(m_count > m_substeps_per_cycle){
    m_count = 0;

    IntegratedSystem::step(h);
  }
}
```

A.4.4 SerializableWrapper

SerializableWrapper adds serializable capabilities: one method to store variables in a stream (it could be easily extended to also store parameters), and one to load them from a stream:

```
void save(std::ostream &os) const
{
  for(int i = 0; i < System::n_variables; i++){
    os << System::m_variables[i] << " ";
  }
}

void load(std::istream &is)
{
  for(int i = 0; i < System::n_variables; i++){
    is >> System::m_variables[i];
  }
}
```

Bibliography

[1] G. Orlovsky, T. G. Deliagina, and S. Grillner, *Neuronal Control of Locomotion*. Oxford university press, November 1999.

[2] C. Tin and C.-S. Poon, "Internal models in sensorimotor integration: perspectives from adaptive control theory," *Journal of Neural Engineering*, vol. 2, pp. S147+, September 2005.

[3] N. G. Hatsopoulos, "Coupling the neural and physical dynamics in rhythmic movements.," *Neural Comput*, vol. 8, pp. 567–581, April 1996.

[4] A. I. Selverston, M. I. Rabinovich, H. D. Abarbanel, R. Elson, A. Szucs, R. D. Pinto, R. Huerta, and P. Varona, "Reliable circuits from irregular neurons: A dynamical approach to understanding central pattern generators," *Journal of Physiology-Paris*, vol. 94, pp. 357–374, December 2000.

[5] R. C. Elson, A. I. Selverston, H. D. Abarbanel, and M. I. Rabinovich, "Inhibitory synchronization of bursting in biological neurons: Dependence on synaptic time constant," *J Neurophysiol*, vol. 88, pp. 1166–1176, September 2002.

[6] C. Van Vreeswijk, L. F. Abbott, and B. G. Ermentrout, "When inhibition not excitation synchronizes neural firing," *Journal of Computational Neuroscience*, vol. 1, pp. 313–321, December 1994.

[7] P. Varona, M. I. Rabinovich, A. I. Selverston, and Y. I. Arshavsky, "Winnerless competition between sensory neurons generates chaos: A possible mechanism for molluscan hunting behavior," *Chaos: An Interdisciplinary Journal of Nonlinear Science*, vol. 12, no. 3, pp. 672–677, 2002.

[8] F. B. Rodríguez, P. Varona, R. Huerta, M. I. Rabinovich, and H. D. I. Abarbanel, "Richer network dynamics of intrinsically non-regular neurons measured through mutual information," in *Connectionist Models of Neurons, Learning Processes and Artificial Intelligence : 6th International Work-Conference on Artificial and Natural Neural Networks, IWANN 2001 Granada, Spain, June 13-15, 2001, Proceedings, Part I*, vol. 2084/2001, pp. 490+, Springer Berlin / Heidelberg, 2001.

[9] F. K. Skinner, N. Kopell, and E. Marder, "Mechanisms for oscillation and frequency control in reciprocally inhibitory model neural networks," *Journal of Computational Neuroscience*, vol. 1, June 1994.

[10] X.-J. Wang and J. Rinzel, "Alternating and synchronous rhythms in reciprocally inhibitory model neurons," *Neural Comput.*, vol. 4, pp. 84–97, January 1992.

[11] G. A. Bekey, *Autonomous Robots, From Biological Inspiration to Implementation and Control*, vol. 24. New York, NY, USA: MIT Press, March 2005.

[12] R. Huerta, P. Varona, M. I. Rabinovich, and H. D. Abarbanel, "Topology selection by chaotic neurons of a pyloric central pattern generator.," *Biol Cybern*, vol. 84, January 2001.

[13] J. González-Gómez, E. Aguayo, and E. Boemo, "Locomotion of a modular worm-like robot using a fpga-based embedded microblaze soft-processor," in *7th International Conference on Climbing and Walking Robots*, September 2004.

[14] A. Kamimura, H. Kurokawa, E. Toshida, K. Tomita, S. Murata, and S. Kokaji, "Automatic locomotion pattern generation for modular robots," in *IEEE International Conference on Robotics and Automation*, vol. 1, pp. 714–720 vol.1, 2003.

[15] A. Ijspeert and J. Kodjabachian, "Evolution and development of a central pattern generator for the swimming of a lamprey," 1998.

[16] J. Ayers, C. Wilbur, and C. Olcott, "Lamprey robots," in *International Symposium on Aqua Biomechanisms* (T. Wu and N. Kato, eds.), Tokai University, 2000.

[17] P. Manoonpong, T. Geng, T. Kulvicius, B. Porr, and F. Wörgötter, "Adaptive, fast walking in a biped robot under neuronal control and learning," *PLoS Computational Biology*, May 2007.

[18] C. Koch, *Biophysics of Computation: Information Processing in Single Neurons (Computational Neuroscience)*. Oxford University Press, November 1998.

[19] C. Aguirre, D. Campos, P. Pascual, and E. Serrano, "A model of spiking-bursting neuronal behavior using a piecewise linear two-dimensional map," pp. 130–135, 2005.

[20] E. M. Izhikevich and F. Hoppensteadt, "Classification of bursting mappings," *International Journal of Bifurcation and Chaos*, vol. 14, no. 11, pp. 3847–3854, 2004.

[21] N. F. Rulkov, "Modeling of spiking-bursting neural behavior using two-dimensional map," *Physical Review E*, vol. 65, pp. 041922+, April 2002.

[22] A. Destexhe, Z. F. Mainen, and T. J. Sejnowski, "An efficient method for computing synaptic conductances based on a kinetic model of receptor binding," *Neural Comput.*, vol. 6, pp. 14–18, January 1994.

[23] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns*. Addison-Wesley Professional, January 1995.

[24] J. Siek and A. Lumsdaine, "Concept checking: Binding parametric polymorphism in c++," in *Proceedings First Workshop on C++ Template Programming*, (Erfurt, Germany), 2000.

[25] J. O. Coplien, "Curiously recurring template patterns," *C++ Rep.*, vol. 7, pp. 24–27, February 1995.

[26] Y. Smaragdakis and D. Batory, "Mixin-based programming in c++," 2000.

[27] U. W. Eisenecker, F. Blinn, and K. Czarnecki, "A solution to the constructor-problem of mixin-based programming in c++," in *First Workshop on C++ Template Programming, Erfurt, Germany*, October 2000.

[28] W. B. Kristan, "Neuroscience: A push-me pull-you neural design," *Science*, vol. 315, pp. 339–340, January 2007.

[29] J. L. Hindmarsh and R. M. Rose, "A model of neuronal bursting using three coupled first order differential equations," *Proceedings Of The Royal Society Of London. Series B, Containing Papers Of a Biological Character. Royal Society (Great Britain)*, vol. 221, pp. 87–102, March 1984.